FPGA Prototyping with VHDL
Learning by doing

Manuel G. Gericota

Department of Electrical Engineering
School of Engineering – Polytechnic of Porto
&
Fakultet for teknologi, naturvitenskap og maritime fag
Institutt for realfag og industrisystemer
Universitetet i Sørøst-Norge
Kongsberg, Norway

VHDL Standards

- IEEE Standard VHDL Language Reference Manual
  - IEEE Std 1076-1987
  - IEEE Std 1076-1993
  - IEEE Std 1076-2000
  - IEEE Std 1076-2002
  - IEEE Std 1076-2008

  + several additions → Mathematical Packages, Synthesis Packages, Application-Specific Integrated Circuit (ASIC) Modeling Specification, Analog and Mixed-Signal Extensions, Register Transfer Level (RTL) Synthesis
- IEEE Standard Multivalue Logic System for VHDL Model Interoperability
  - IEEE Std 1164-1993

Learning by doing

- VHDL is intended for describing and modeling a digital system at various levels; it is an extremely complex language
- VHDL ⟶ general and flexible ⟶ the same circuit may be described by a wide variety of language constructs and coding styles
  - avoid unnecessary complex implementations
  - do not write no synthesizable code

Learning by doing

- Template approach
  - forces us to think more about hardware and to develop a good coding practice for synthesis
- Our focus is on hardware design rather than on the language

VHDL design descriptions

VHDL design description = ENTITY declaration + ARCHITECTURE body

- ENTITY declaration ⟶ design I/O
- ARCHITECTURE body ⟶ content or functionality of the entity
  - Modeling styles
    - Structural – the entity is described as a set of interconnected components
    - Functional or Dataflow – the flow of data through the entity is expressed using concurrent signal assignment statements (the structure of the entity is not explicitly specified, but it can be implicitly deduced)
    - Behavioral – the behavior of an entity as a set of statements that are executed sequentially in the specified order
  - Every architecture needs an entity so it is common to refer to them together as an ENTITY/ARCHITECTURE PAIR

Our first design

- One-bit comparator

input		output
i0	i1	eq
0	0	1
0	1	0
1	0	0
1	1	1

- One way to describe the circuit is to use a sum-of-products format

$$eq = \overline{i0} \cdot \overline{i1} + i0 \cdot i1$$

Gate-level description of a 1-bit comparator

Ex. 1

```
library ieee;
use ieee. std_logic_1164. all ;

entity eq1 is
    port (
          i0, i1: in std_logic;
          eq: out std_logic
    ) ;
end eq1;
```

Identifier

VHDL keywords are in **bold**

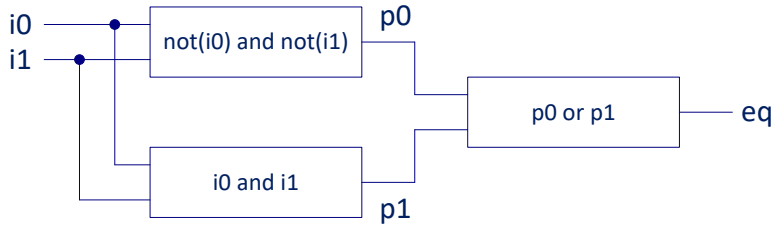
```
architecture sop_arch of eq1 is
    signal p0, p1: std_logic;

begin
    -- sum of two product terms
    eq <= p0 or p1;
    -- product terms
    p0 <= (not i0) and (not i1);
    p1 <= i0 and i1;
end sop_arch;
```

Comment

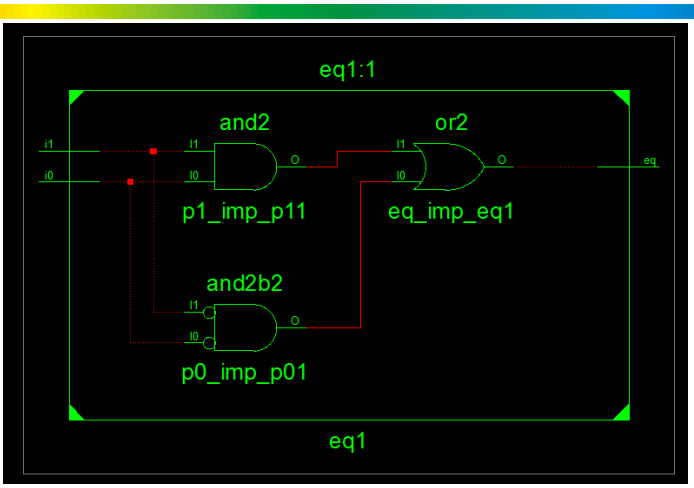
Functional or Dataflow model

Graphical representation of the gate-level description of a 1-bit comparator

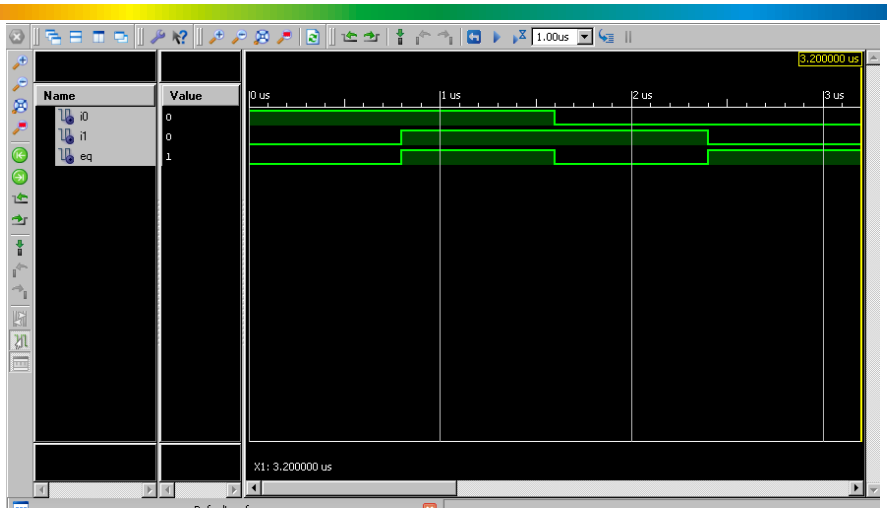


- Three circuit parts ⟶ three concurrent statements
- The connections among these parts are implicitly specified by the signal and port names
- The order of the concurrent statements is clearly irrelevant and the statements can be rearranged arbitrarily

Gate-level implementation of a 1-bit comparator



Logic simulation of the 1-bit comparator



Basic lexical rules

- VHDL is
  - case insensitive – upper- and lowercase letters can be used interchangeably
  - free formatting – spaces and blank lines can be inserted freely
  - all VHDL statements finish with a semi-colon
- It is good practice
  - to add proper spaces to make the code clear
  - to associate special meaning with cases

Basic lexical rules

- Identifier
  - name of an object – maximum 26 letters, digits, and the underscore (_); must always start with a letter
- Comments
  - start with '--' and the text after it is ignored
- Library and package

```
library ieee;
use ieee. std_logic_1164. all ;
```

  - invoke the std_logic_1164 package from the `ieee` library
  - packages → add additional types, operators, functions, etc. to VHDL

The **IEEE 1164** standard defines a package design unit that contains declarations that support a uniform representation of a logic value in a VHDL hardware description

Entity declaration

- Lists the I/O port names/signals of the module/circuit → may be associated to physical device pins or not

```
entity eq1 is
   port (
     i0, i1: in std_logic;
     eq: out std_logic
   ) ;
end eq1;
```

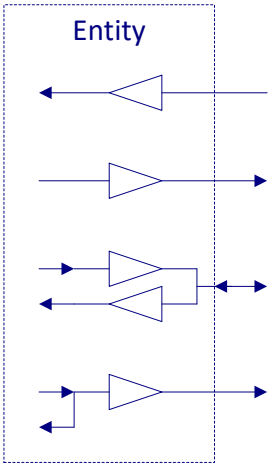
each port name is unique and has an associated mode and data type

  - first line → indicates the name of the module/circuit `eq1`
  - port section → specifies the I/O port names/signals
- the basic format for an I/O port declaration is

```
signal_name1 , signal_name2 , … : mode data_type;
```


Entity declaration

- The port's ***mode*** indicates the direction of the signal/data flow:
  - `IN` → signal/data goes into the entity only
  - `OUT` → signal/data goes out of the entity only (and is not used internally)
  - `INOUT` → signal/data is bi-directional (goes into and out of the entity)
  - `BUFFER` → signal/data that goes out of the entity and is also fed-back internally



Data types

- Pre-Defined Data Types
  - Package standard of library std: BIT, BIT_VECTOR, BOOLEAN, INTEGER, REAL (not synthesizable)
  - Package `IEEE.STD_LOGIC_1164.ALL`: STD_LOGIC, STD_LOGIC_VECTOR, STD_ULOGIC, and STD_ULOGIC_VECTOR
  - Package `IEEE.NUMERIC_STD.ALL`: SIGNED, UNSIGNED, INTEGER, plus several data conversion functions

Binary representation of integers -2^{N-1} to $2^{N-1}-1$

Binary representation of positive integers 0 to 2^N-1

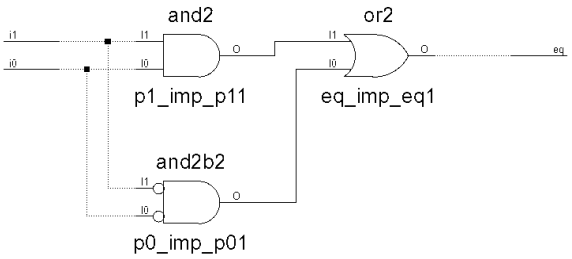
In sum – The Entity declaration

```
ENTITY entity_name IS
   -- optional generics
   PORT (
        name : mode type ;
        ...
   ) ;
   END entity_name;
```

- *entity_name* → an arbitrary name
- *generics* → used for defining customizable components
- *name* → signal/port identifier
- *mode* → indicates the direction the signal/data flow
- *type* → indicates the set of values *name* may be assigned

Architecture body

```
architecture sop_arch of eq1 is
   signal p0, p1: std_logic;
begin
   -- sum of two product terms
   eq <= p0 or p1;
   -- product terms
   p0 <= (not i0) and (not i1);
   p1 <= i0 and i1;
end sop_arch;
```



- Describes the operation of the circuit
  - VHDL allows multiple bodies associated with an entity
    - thus, the body is identified by the name `sop_arch` (sum-of-products architecture)

Architecture body

- It may include an optional declaration section
  - specifies internal signals, constants
  - two internal signals are declared in this description

```
signal p0, p1: std_logic;
```

- The main description encompassed between **`begin`** and **`end`**, contains three concurrent statements
  - signal on the left-hand side of the statement → output of that part
  - the expression specifies the circuit function + corresponding input signals

```
eq <= p0 or p1;
```

  - the statement is activated and the expression evaluated each time `p0` or `p1` changes its value; the new value is assigned to `eq` after the propagation delay

In sum - the Architecture

- Describes the structure or functionality/behavior of entities
- Descriptions can be either a combination of
  - Structural descriptions
    - instantiations (placements of logic-much like in a schematic-and their connections) of building blocks referred to as components
  - Behavioral/Dataflow descriptions
    - Algorithmic (or "high-level") descriptions:

```
IF a = b THEN state <= state5;
```

    - Boolean equations (also referred to as dataflow):

```
x <= a OR (b AND c);
```


In sum - the Architecture declaration

```
ARCHITECTURE arch_name OF entity_name IS
   -- optional signal declarations, constants, etc...
BEGIN
   --VHDL statements
END arch_name;
```

- *arch_name* is an arbitrary name
- optional *signal* and/or *constant* declarations are used for signals and/or constants local to the architecture body (not the entity's I/O or generics).
- *entity_name* is the entity name
- *statements* describe the function or contents of the entity

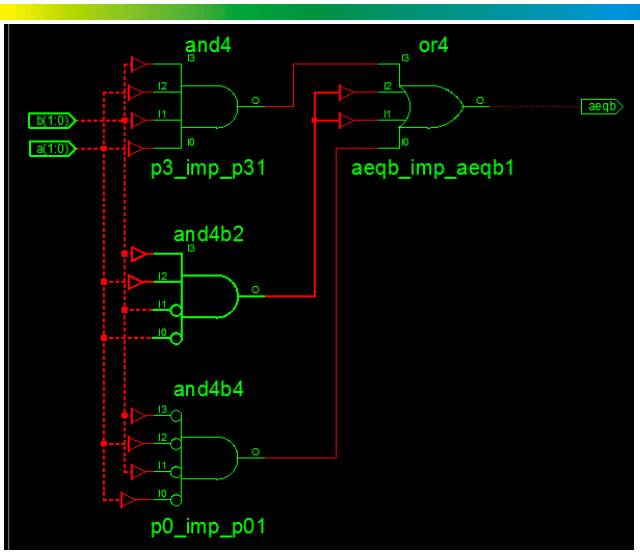
Gate-level implementation of a 2-bit comparator

Ex. 2

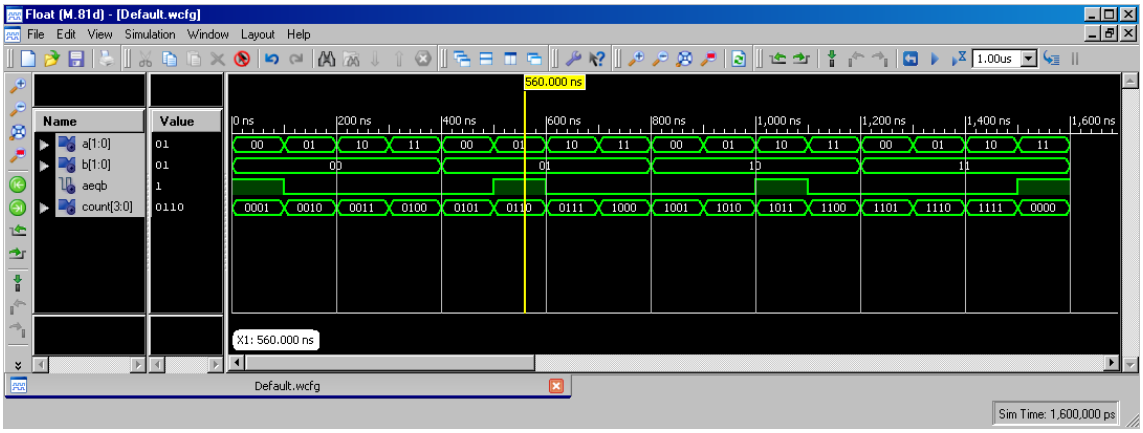
```
library ieee;
use ieee. std_logic_1164. all ;
entity eq2 is
    port (
          a, b: in std_logic_vector(1 downto 0);
          aeqb: out std_logic
    ) ;
end eq2;

architecture sop_arch of eq2 is
    signal p0, p1, p2, p3: std_logic;
begin
    aeqb <= p0 or p1 or p2 or p3; -- sum of four product terms
    -- product terms
    p0 <= ((not a(1)) and (not b(1))) and ((not a(0)) and (not b(0)));
    p1 <= ((not a(1)) and (not b(1))) and (a(0) and b(0));
    p2 <= (a(1) and b(1)) and ((not a(0)) and (not b(0)));
    p3 <= (a(1) and b(1)) and (a(0) and b(0));
end sop_arch;
```


Gate-level implementation of a 2-bit comparator

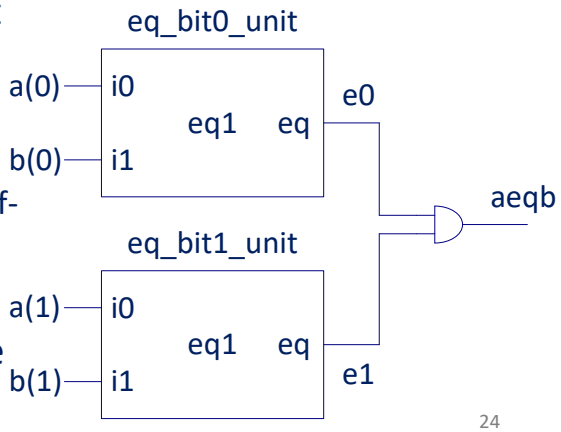


Logic simulation results of the gate-level implementation of a 2-bit comparator



Gate-level implementation of a 2-bit comparator

- a, b are now declared as 2-element `std_logic_vector`
- Architecture body similar to that of the 1-bit comparator
  - `p0`, `p1`, `p2`, and `p3` signals ⇒ results of the 4 product terms
  - `aeqb` ⟶ logic expression in sum-of-products format
- Alternatively ⟶ use of the previous 1-bit comparator as the building block of the 2-bit one



Structural description

- *Instantiation* - enables to build large systems from simpler or predesigned components → *structural description*
- Example of a direct instantiation

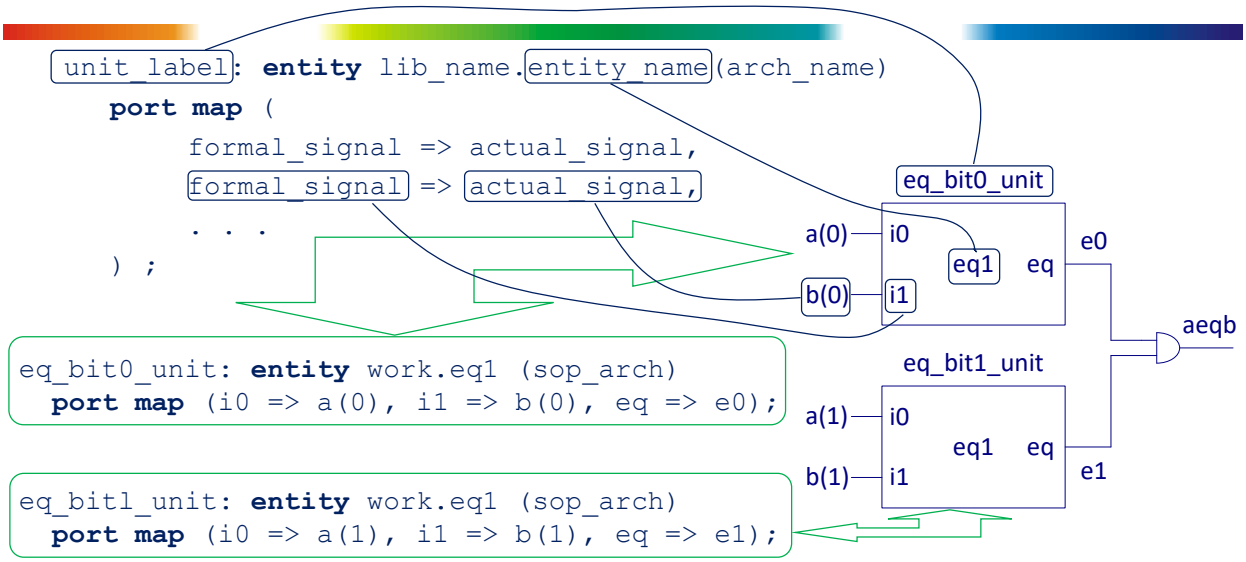
Ex. 3

```
architecture struc_arch of eq2 is
   signal e0, e1: std_logic;
begin
   -- instantiate two 1-bit comparators
   eq_bit0_unit: entity work.eq1 (sop_arch)
        port map (i0 => a(0), i1 => b(0), eq => e0);
   eq_bitl_unit: entity work.eq1 (sop_arch)
        port map (i0 => a(1), i1 => b(1), eq => e1);
   -- a and b are equal if individual bits are equal
   aeqb <= e0 and e1;
end struc_arch;
```

Defines the architecture to be used in the instantiation, as an entity may have more than one architecture

Direct instantiation

```
unit_label: entity lib_name.entity_name(arch_name)
    port map (
         formal_signal => actual_signal,
         formal_signal => actual_signal,
         . . .
    ) ;
```




```
eq_bit0_unit: entity work.eq1 (sop_arch)
  port map (i0 => a(0), i1 => b(0), eq => e0);
```

```
eq_bitl_unit: entity work.eq1 (sop_arch)
  port map (i0 => a(1), i1 => b(1), eq => e1);
```

Component instantiation

- Alternative style ⟶ components are declared before being used

```
architecture struc_arch of eq2 is
signal e0, e1: std_logic;
COMPONENT eq1           -- component instantiation
    PORT(
         i0, i1 : IN  std_logic;
         eq : OUT  std_logic
        );
END COMPONENT;
begin                   -- instantiate two 1-bit comparators
    eq_bit0_unit: eq1
       port map (i0 => a(0), i1 => b(0), eq => e0);
    eq_bit1_unit: eq1
       port map (i0 => a(1), i1 => b(1), eq => e1);
    aeqb <= e0 and e1;
end struc_arch;
```

It is assumed that there is only one architecture for the component

Ex. 3a

Instantiation

- If a particular output port is not connected or used ⟶ `OPEN` keyword is used in the PORT MAP association list

  e.g.: `port map(i0 => a(1), i1 => b(1), eq => open);`

  It cannot be used for an input port ⟶ input ports cannot be left floating
- Not used inputs ⇒ necessary to define a neutral value, like in maths
- The neutral value must not interfere with the logical result imposed by the other inputs that are effectively being used
- e.g. if we have a AND gate with four inputs but we are using only three, the remaining fourth must be set to logic '1', i.e.

  port map (i0 => a(1), i1 => b(1), i2 => c(1), i3 =>'1', eq => out);

Instantiation

- `work library` → default library in which the compiled entity and architecture units are stored
- `eq1` and `sop_arch` → names of the `entity` and `architecture` defined in the gate-level implementation of the 1-bit comparator
- The component instantiation is also a concurrent statement
- The code is a textual description of a schematic → **Structural description**

Modeling with a process

- VHDL contains a number of sequential statements, which are executed in sequence
  - since their behavior is different from that of a normal concurrent circuit model, these statements are encapsulated inside a `process`
  - a `process` in itself is a concurrent statement
- Sequential statements include a rich variety of constructs → many of them cannot be synthesized at all
- Synthesizable processes include
  - describing routing structures with `if` and `case` statements
  - constructing templates for memory elements

Sensitivity list

- The *sensitivity_list* is a list of signals to which the `process` responds (i.e., is "sensitive to")
- The body of a process is composed of any number of sequential statements

Syntax of a process

- The simplified syntax of a process with a sensitivity list is

```
process (sensitivity_list)
begin
    sequential statement;
    sequential statement;
    ...
end process;
```

Sequential signal assignment statement

- *sequential* signal assignment statement → the simplest sequential statement
  - the simplified syntax is

    ```
    sig <= value_expression;
    ```
  - the statement must be encapsulated inside a `process`
  - syntax → similar to that of a simple *concurrent* signal assignment statement **But** the semantics are different
  - when a signal is assigned multiple times inside a process, only the last assignment takes effect

Sequential signal assignment statement

```
process (a , b)
begin
     c <= a and b;
     c <= a or b;
end process;
```

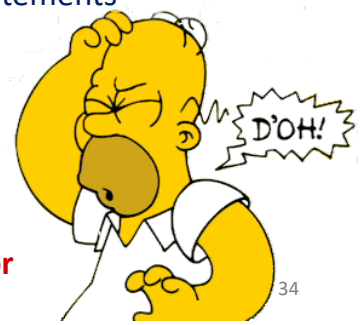
⟺

```
process (a , b)
begin
   c <= a or b;
end process;
```

- **But** if they are concurrent signal assignment statements

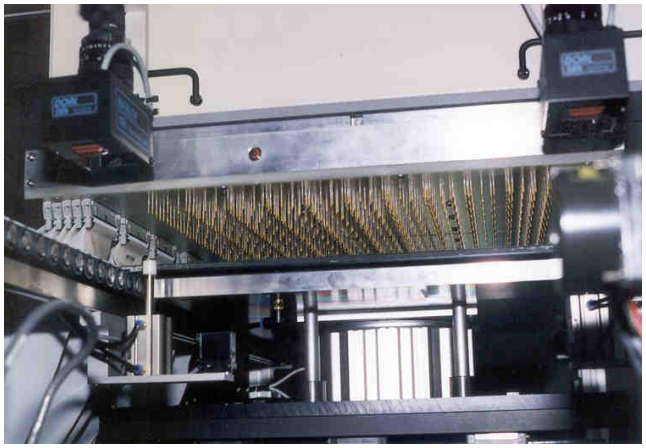
```
-- not within a process
c <= a and b;
c <= a or b;
```

- the code infers an *and* cell and an *or* cell, whose outputs are tied together → this is not allowed in most device technology and thus it is a **design error**



Test bench

- PCBs where tested using a bed-of-nails

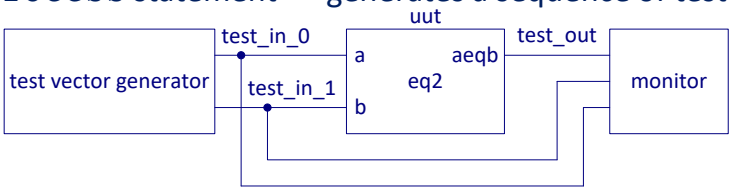


Creating test benches

- Simulation
  - after development, the description must be simulated in a host computer to verify the correctness of the circuit operation
  - simulation is usually performed within the same HDL framework
    - graphical
    - using a special VHDL construct
- Test bench
  - a special program that mimic a physical lab bench
  - **in the test bench,** operations are performed **sequentially**

Test bench for the 2-bit comparator

- Sketch of a 2-bit comparator test bench program
  - the *uut* is the *unit under test*
  - the *test vector generator* block generates testing input patterns
  - the *monitor* block examines the output responses
- The code consists of
  - a component instantiation statement ⟶ creates an instance of the 2-bit comp
  - a `process` statement ⟶ generates a sequence of test patterns



Writing test benches with VHDL

```
library ieee;
use ieee.std_logic_1164.all;

entity eq2_testbench is
end eq2_testbench;

architecture tb_arch of eq2_testbench is

   signal test_in0, test_in1: std_logic_vector (1 downto 0);
   signal test_out: std_logic;

begin

   -- instantiate the circuit under test

   uut: entity work.eq2 (struc_arch)
        port map (a=>test_in0, b=>test_in1, aeqb=>test_out);
```

No I/O ports → the unit-under-test is 'submerged' into the test bench entity

Ex. 4

Writing test benches with VHDL

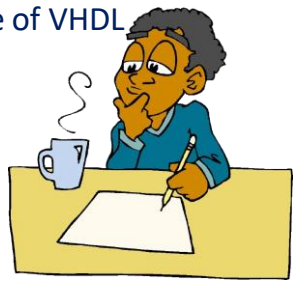
```
-- test vector generator
process
begin
     -- test vector 1
     test_in0 <= "00";
     test_in1 <= "00";
     wait for 200 ns;
     -- test vector 2
     test_in0 <= "01";
     test_in1 <= "00";
     wait for 200 ns;
     -- test vector 3
     test_in0 <= "01";
     test_in1 <= "11";
     wait for 200 ns;
```

- Each test pattern comprises three statements
  - values assigned to `test_in0` and `test_in1` signals
  - duration of the assignment 200 ns
- The code has no monitor
  - input and output waveforms are observed on a simulator's display → a kind of "virtual logic analyzer"

Writing test benches with VHDL

```
     -- test vector 4
     test_in0 <= "10";
     test_in1 <= "10";
     wait for 200 ns;
     -- test vector 5
     test_in0 <= "10";
     test_in1 <= "00";
     wait for 200 ns;
     -- test vector 6
     test_in0 <= "11";
     test_in1 <= "11";
     wait for 200 ns;
     -- test vector 7
     test_in0 <= "11";
     test_in1 <= "01";
     wait for 200 ns;
     wait;
  end process;
end tb_arch;
```

- Writing code for a comprehensive test vector generator and a monitor requires deep knowledge of VHDL



- Terminates simulation

More about IEEE 1076 data types

- VHDL is a strongly typed language rich in data types
  - you cannot assign a signal of one type to a signal of another type
  - only a small set of predefined types are suitable for synthesis
- Synthesizable data types – scalars and vectors
  - The predefined synthesizable types in each of these categories are:
    - Scalars: BIT, STD_LOGIC, STD_ULOGIC, and BOOLEAN
    - Vectors: BIT_VECTOR, STD_LOGIC_VECTOR, STD_ULOGIC_VECTOR, INTEGER, SIGNED, and UNSIGNED

STD_LOGIC (and STD_LOGIC_VECTOR)

- STD_LOGIC (and STD_LOGIC_VECTOR): 8-valued logic system introduced in the IEEE 1164 standard
  - '0' Forcing Low (synthesizable logic '0')
  - '1' Forcing High (synthesizable logic '1')
  - 'Z' High impedance (synthesizable tri-state buffer)
  - 'X' Forcing Unknown
  - 'W' Weak unknown
  - 'L' Weak low
  - 'H' Weak high
  - '–' Don't care

synthesizable

may be found in simulation

not synthesizable

STD_ULOGIC and STD_ULOGIC_VECTOR plus 'U' Unresolved (not synthesizable)

Signals with multiple bits

- `std_logic_vector` data type → array of `std_logic` elements

```
a: in std_logic_vector (7 downto 0);
```

  - `a(7 downto 4)` specifies a desired range
  - `a(1)` enables the access to a single element of the array
- The array can also be declared in ascending order

```
a: in std_logic_vector (0 to 7);
```

  - this format should be avoided since it is more natural to associate the MSB with the leftmost position

STD_LOGIC (and STD_LOGIC_VECTOR)

- e.g.

```
SIGNAL x: STD_LOGIC;
-- x is declared as a one-digit signal of type STD_LOGIC
SIGNAL y: STD_LOGIC_VECTOR (3 DOWNTO 0);
-- y is a 4-bit vector, with the leftmost bit being the MSB
```

- Based on the signals above, the following assignments would be legal (to assign a value to a signal, the "<=" operator must be used):

```
x <= '1';
-- x a single-bit signal, whose value is '1'
-- Single quotes (' ') are used for a single bit
y <= "0111";
-- y a 4-bit signal, whose value is "0111" (MSB='0')
-- Double quotes (" ") are used for vectors
```

BIT (and BIT_VECTOR): 2-level logic ('0', '1')

- Like STD_LOGIC but with only 2 values – '0' and '1'
  - e.g.

```
SIGNAL x: BIT; -- x is a 1-digit signal of type BIT
SIGNAL y: BIT_VECTOR (3 DOWNTO 0);
-- y is a 4-bit vector, with the leftmost bit being the MSB
```



  - Based on the signals above, the following assignments would be legal (to assign a value to a signal, the "<=" operator must be used):

```
x <= '1';
-- x a single-bit signal, whose value is '1'
y <= "0111";
-- y a 4-bit signal, whose value is "0111" (MSB='0')
```


BOOLEAN, INTEGER and NATURAL

- BOOLEAN: True, False
- INTEGER: 32-bit integers
  (from -2,147,483,647 to +2,147,483,647)
- NATURAL: Non-negative integers
  (from 0 to +2,147,483,647)
  - NATURAL is a sub-type of INTEGER

SIGNED and UNSIGNED types

- SIGNED and UNSIGNED data types defined in the `IEEE.NUMERIC_STD.ALL` → like STD_LOGIC_VECTOR, but accept arithmetic operations typical of INTEGER data types
- e.g.

```
x0 <= '0'; -- bit, std_logic, or std_ulogic value '0'
x1 <= "00011111"; -- bit_vector, std_logic_vector,
                  -- std_ulogic_vector, signed, or unsigned
x2 <= "101111"    -- binary representation of decimal 47
x3 <= B"101111" or x3 <= B"0010_1111"
-- binary representation of decimal 47; the underscore allows
-- an easier visualization
```


More data types examples

```
x4 <= O"57" or x4 <= O"5_7" -- octal representation of
                            -- decimal 47
x5 <= X"2F" or x5 <= X"2_F" -- hexadecimal representation
                            -- of decimal 47
n <= 1200;          -- integer
m <= 1_200;         -- integer, underscore allowed
IF ready THEN...    -- Boolean, executed if ready=TRUE
y <= 1.2E-5;        -- real, not synthesizable
q <= d after 10 ns; -- physical, not synthesizable
```

Legal and illegal operations between data of different types

```
SIGNAL a: STD_LOGIC;

SIGNAL b: STD_LOGIC_VECTOR(7 DOWNTO 0);

SIGNAL c: INTEGER RANGE 0 TO 255;

a <= b(5); -- legal (same scalar type: STD_LOGIC)

b(0) <= a; -- legal (same scalar type: STD_LOGIC)

c <= a;    -- illegal (type mismatch: INTEGER x STD_LOGIC)

c <= b;    -- illegal (type mismatch: INTEGER x

           -- STD_LOGIC_VECTOR)
```


Binary signed number representation 2's complement

- Negative signed values follow the 2's complement rule

Bits	Unsigned value	Signed value 2's complement value
0111 1111	127	127
0111 1110	126	126
0000 0010	2	2
0000 0001	1	1
0000 0000	0	0
1111 1111	255	−1
1111 1110	254	−2
1000 0010	130	−126
1000 0001	129	−127

0111 1110	126
1000 0001	1's complement (invert bits)
+ 1	
1000 0010	-126 2's complement

Why would I need to use a signed or unsigned type in place of a `std_logic_vector` type?

- The `std_logic_vector` type should not be used to define a numerically meaningful signal or variable, a signal that looks and behaves like a number
- The `std_logic_vector` type should be only employed for defining "bags of bits"
  - The use of `signed`/`unsigned` types is desirable any time your bags of bits (signals, variables or constants) stop being "bags" and become numbers of type signed, unsigned or even integers

from: Mealy, B., Tappero, F., *Free Range VHDL Book,* freerangefactory.org, 2016

Warning

- Signals/ports types in entities should always be defined as `std_logic` or `std_logic_vector`
  - `std_logic_vector` type defines a precise order for the signals inside the vector
  - other types are not so strict, which may lead to problems when interconnecting different components
- The use of `std_logic` is a good practice, albeit not mandatory

User-defined data types

- VHDL allows the user to define his/her own data types
- Two categories of user-defined data types: integer and enumerated
  - User-defined *integer* types:

```
TYPE my_integer IS RANGE -32 TO 32;
-- A user-defined subset of integers
```

  - User-defined *enumerated* types:

```
TYPE state IS (idle, forward, backward, stop);
-- An enumerated data type, typical of finite state machines
TYPE color IS (red, green, blue, white);
-- Another enumerated data type
```


User-defined data subtypes

- `subtype` → another way of defining data type ranges
- e.g.
  - User-defined *integer* subtype:

```
subtype MY_INTEGER is INTEGER range 48 to 156 ;
```

  - User-defined subtype of a user-defined type:

```
type DIGIT is ('0', '1', '2', '3', '4', '5', '6',
       '7', '8', '9') ;
subtype MIDDLE is DIGIT range '3' to '7' ;
```

Type conversion

- `std_logic_vector`, `unsigned` and `signed` → different data types
- A conversion function is needed to convert signals of different data types

Data type of a	To data type	Conversion function
unsigned, signed	std_logic_vector	std_logic_vector (a)
signed, std_logic_vector	unsigned	unsigned (a)
unsigned, std_logic_vector	signed	signed (a)
unsigned, signed	integer	to_integer (a)
natural	unsigned	to_unsigned (a, size)
integer	signed	to_signed (a, size)

Type conversion

- `std_logic_vector` data type is not interpreted as a number ⇒ cannot be converted directly to an integer, and vice-versa

```
library ieee;
use ieee. std_logic_1164. all;
use ieee. numeric_std. all ;
signal s1, s2, s3, s4, s5, s6: std_logic_vector (3 downto 0);
signal u1, u2, u3, u4, u5, u6: unsigned (3 downto 0);
```



```
u1 <= s1; -- not ok, type mismatch (unsigned, std_logic_vector)
u2 <= 5;  -- not ok, type mismatch (unsigned, natural)
s2 <= u3; -- not ok, type mismatch (std_logic_vector, unsigned)
s3 <= 5;  -- not ok, type mismatch (std_logic_vector, natural)
```

- The right-hand-side expression → converted to the left-hand-side signal

Type conversion



```
u1 <= unsigned (s1);          -- ok, type match
u2 <= to_unsigned (5, 4);     -- ok, type match size 4 -> 4 bits
                              -- are needed to represent number 5
```



```
s2 <= std_logic_vector (u3);                  -- ok, type match
s3 <= std_logic_vector (to_unsigned (5, 4));  -- ok, type match
```

- Two types of conversion are needed to the last statement; no direct conversion available

RT (Register Transfer) -level combinational circuit

- HDL description of module-level circuits
  - composed of intermediate-sized components, such as adders, comparators, and multiplexers
  - components ⟶ basic building blocks used in *register transfer methodology*
  - RT methodology ⟶ a design technique ⟹ describes system operation by a sequence of data transfers and manipulations among registers ⟶ dataflow
  - supports the variables and sequential execution of an algorithm and provide a systematic way to convert an algorithm into hardware

RT-level components

- Synthesizable operators
  - logical operators
  - relational operators
  - several arithmetic operators

Logical operators

- Operators and data types

Operator	Description	Data type of operands	Data type of result
not a	negation	std_logic, std_logic_vector	same type as operand(s)
a **and** b	and		
a **nand** b	nand		
a **or** b	or		
a **nor** b	nor		
a **xor** b	xor (exor)		
a **xnor** b	xnor (exnor)		

bit-wise operation → operations are executed on bit vectors at the level of their individual bits

Relational operators

- Operators and data types

Operator	Description	Data type of operands	Data type of result
a = b	equal to	any	boolean
a /= b	not equal to		
a < b	less than	scalar or 1-D	
a <= b	less than or equal to		
a > b	greater than		
a >= b	greater than or equal to		

Arithmetic operators

(known as operator overloading under `numeric_std`)

- Operators and data types

Operator	Description	Data type of operands	Data type of result
a + b	addition	unsigned, signed, integer	same type as operands
a - b	subtraction		
a * b	multiplication		
a / b	division		
a REM b	remainder		
a MOD b	modulo		

Arithmetic operators

(known as operator overloading under `numeric_std`)

- Operators and data types

Operator	Description	Data type of operands	Data type of result
a + b	addition	unsigned, signed, integer	same type as operands
a - b	subtraction		
a * b	multiplication		
a / b	division		
a REM b	remainder		
a MOD b	modulo		

Integer division → division in which the fractional part (remainder) is discarded

Weisstein, Eric W. "Integer Division." From MathWorld--A Wolfram Web Resource. http://mathworld.wolfram.com/IntegerDivision.html

Arithmetic operators

(known as operator overloading under `numeric_std`)

- Operators and data types

Operator	Description	Data type of operands	Data type of result
a + b	addition	unsigned, signed, integer	same type as operands
a - b	subtraction		
a * b	multiplication		
a / b	division		
a REM b	remainder		
a MOD b	modulo		

The quantity (number) left over when two integers are divided by each other in integer division

Weisstein, Eric W. "Remainder." From MathWorld--A Wolfram Web Resource. http://mathworld.wolfram.com/Remainder.html

Arithmetic operators

(known as operator overloading under `numeric_std`)

- Operators and data types

Operator	Description	Data type of operands	Data type of result
a + b	addition		
a - b	subtraction		
a * b	multiplication		
a / b	division		
a REM b	remainder		
a MOD b	modulo		

If two numbers a and c have the property that their difference a-c is integrally divisible by a number b (i.e., *(a-c)/b* is an integer), then a and c are said to be "congruent modulo b". The number b is called the modulus, and the statement "a is congruent to c (modulo b)" is written mathematically as

c = a (mod b)

Weisstein, Eric W. "Congruence." From MathWorld--A Wolfram Web Resource. http://mathworld.wolfram.com/Congruence.html

Arithmetic operators

(known as operator overloading under `numeric_std`)

- Operators and data types

Operator	Description	D
a + b	addition	u
a - b	subtraction	ir
a * b	multiplication	
a / b	division	
a REM b	remainder	
a MOD b	modulo	

e.g.

5 rem 3 = 2
5 mod 3 = 2
(-5) rem 3 = -2
(-5) mod 3 = 1
(-5) rem (-3) = -2
(-5) mod (-3) = -2
5 rem (-3) = 2
5 mod (-3) = -1

"IEEE Standard VHDL Language Reference Manual," in IEEE Std 1076-2008 (Revision of IEEE Std 1076-2002) , pp.c1-626, Jan. 26 2009

Arithmetic operations vs types

- Statements involving arithmetic operations



```
u4 <= u2 + ul; -- ok, both operands unsigned
u5 <= u2 + 1;  -- ok, operands unsigned and natural
```

  - both statements are valid since the + operator is defined with the unsigned and natural types in the IEEE numeric_std package



```
s5 <= s2 + sl; -- not ok, + undefined over the types
s6 <= s2 + 1;  -- not ok, + undefined over the types
```

  - both statements are invalid since the + operator is not defined within the `std_logic_vector` data type in the IEEE numeric_std package

Arithmetic operations vs types

- to perform the operation operands must be converted to the unsigned (or signed) data type, perform addition, and then convert the result back to the std_logic_vector data type



```
s5 <= std_logic_vector(unsigned(s2) + unsigned(sl)); -- ok
s6 <= std_logic_vector(unsigned(s2) + 1);            -- ok
```

Nonstandard arithmetic packages

- There are several non-IEEE arithmetic packages → `std_logic_arith` `std_logic_unsigned` `std_logic_signed`
  - the `std_logic_arith` package is similar to `numeric_std`
  - the other two packages do not introduce any new data type but define overloaded arithmetic operators over `std_logic_vector` data type
  - this approach eliminates the need for data conversion



Although using these packages seems to be less cumbersome initially, it is not a good practice since these packages are not a part of IEEE standards and may introduce a compatibility problem in the long run



Warning

- It is a good practice to avoid the proliferation of types in the same design
- Use conversion functions whenever necessary
- Do not complicate the designs unnecessarily



Shift Operators
(VHDL 2008 only)

- Operators and data types

Operator	Description	Data type of left operand	Data type of right operand	Data type of result
a sll n	shift left logical	std_logic_vector, unsigned, signed	integer	same as left operand
a srl n	shift right logical			
a rol n	rotate left			
a ror n	rotate right			
a sla n	shift left arithmetic	unsigned, signed		
a sra n	shift right arithmetic			

Shift Operators
(VHDL 2008 only)

- Operators and data types

Operator	Description	Data type of left operand	Data type of right operand	Data type of result
a sll n	shift left logical	std_logic_vector,	integer	same as left
a srl n	shift right logical			
a rol n	rotate left			
a ror n	rotate right			
a sla n	shift left arithmetic	unsigned, signed		
a sra n	shift right arithmetic			

a is shifted left by **n** bits and filled on the right with '0's (logic shift)

Shift Operators
(VHDL 2008 only)

- Operators and data types

Operator	Description	Data type of left operand	Data type of right operand	Data type of result
a sll n	shift left logical	std_logic_vector, unsigned, signed	integer	same as left
a srl n	shift right logical			
a rol n	rotate left			
a ror n	rotate right			
a sla n	shift left arithmetic	unsigned, signed		
a sra n	shift right arithmetic			

a is shifted right by **n** bits and filled on the left with '0's (logic shift)

Shift Operators
(VHDL 2008 only)

- Operators and data types

Operator	Description	Data type of left operand	Data type of right operand	Data type of result
a sll n	shift left logical	std_logic_vector,	integer	same as
a srl n	shift right logical			
a rol n	rotate left			
a ror n	rotate right			
a sla n	shift left arithmetic	unsigned, signed		
a sra n	shift right arithmetic			

a is rotated left by **n** bits; the **n** bits that were shifted off the left end are copied to the right of the shifted **a** bits

Shift Operators
(VHDL 2008 only)

- Operators and data types

Operator	Description	Data type of left operand	Data type of right operand	Data type of result
a sll n	shift left logical	std_logic_vector,	integer	same as
a srl n	shift right logical			
a rol n	rotate left			
a ror n	rotate right			
a sla n	shift left arithmetic	unsigned, signed		
a sra n	shift right arithmetic			

a is rotated right by **n** bits; the **n** bits that were shifted off the right end are copied to the left of the shifted **a** bits

Shift Operators
(VHDL 2008 only)

- Operators and data types

Operator	Description	Data type of left operand	Data type of right operand	Data type of result
a sll n	shift left logical	std_logic_vector,	integer	same as
a srl n	shift right logical			
a rol n	rotate left			
a ror n	rotate right			
a sla n	shift left arithmetic	unsigned, signed		
a sra n	shift right arithmetic			

a is shifted left by **n** bits and filled on the right with the value of the LSB (arithmetic shift)

Shift Operators
(VHDL 2008 only)

- Operators and data types

Operator	Description	Data type of left operand	Data type of right operand	Data type of result
a sll n	shift left logical	std_logic_vector,	integer	same as
a srl n	shift right logical			
a rol n	rotate left			
a ror n	rotate right			
a sla n	shift left arithmetic	u…ed, signed		
a sra n	shift right arithmetic			

a is shifted right by **n** bits and filled on the left with the value of the MSB (arithmetic shift)

Shift Operators
(VHDL 2008 only)

- Operators and data types

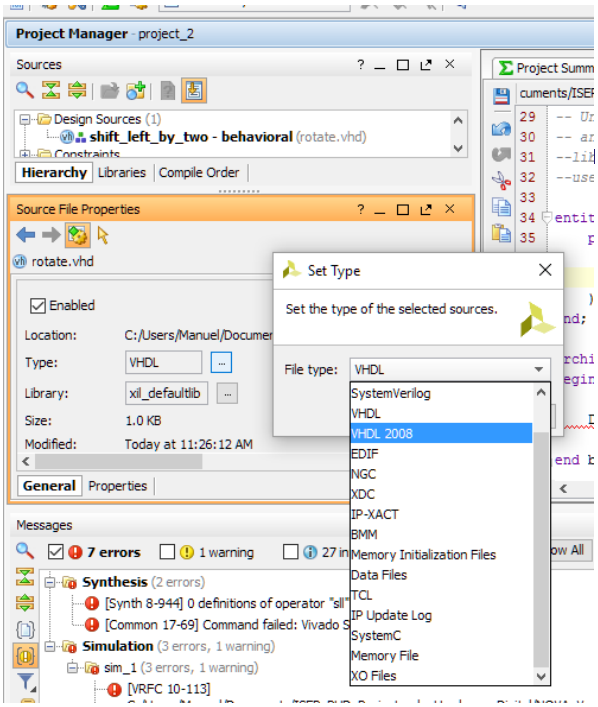
Operator	Description	Data type of left operand	Data type of	Data type of result
a sll n	shift left logical	std_…		…s
a srl n	shift right logical	u…		
a rol n	rotate left			
a ror n	rotate right			
a sla n	shift left arithmetic	unsign…		
a sra n	shift right arithmetic			

For all the operators, if **n** is negative then **a** is shifted or rotated for the opposite side from that defined by the operator

AVOID

Shift Operators
(VHDL 2008 only)

- Setting up Vivado to use VHDL-2008
  - Source File Properties window
  - Set Type: VHDL 2008



Concatenation operator

- The operator `&` (ampersand)
  - combines segments of elements and small arrays to form a large array
    - (k_1+k_2)-bit word <= k_1-bit word & k_2-bit word
  - e.g.

```
signal a1: std_logic;
signal a4: std_logic_vector (3 downto 0);
signal b8, c8, d8: std_logic_vector (7 downto 0);
...
b8 <= a4 & a4;
c8 <= a1 & a1 & a4 & "00";
d8 <= b8 (3 downto 0) & c8 (3 downto 0);
```

The operands can be of any of the types listed for logical operators, like std_logic and std_logic_vector

Concatenation operator

- the operator & may be used in shifting operations

```
signal a: std_logic_ vector (7 downto 0);
signal rot, shl, sha: std_logic_ vector (7 downto 0);

-- rotate a to right 3 bits
rot <= a (2 downto 0) & a (7 downto 3);

-- shift a to right 3 bits and insert 0 (logic shift)
shl <= "000" & a (7 downto 3);

-- shift a to right 3 bits and insert MSB
-- (arithmetic shift)
sha <= a(7) & a(7) & a(7) & a(7 downto 3);
```


Concatenation operator

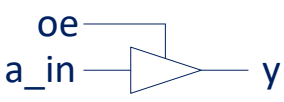
- Other uses of the & operator
  - to adjust vector sizes
  - e.g.

```
signal a, b : UNSIGNED (7 downto 0);
signal sum : UNSIGNED (8 downto 0);

sum <= ('0' & a) + ('0' & b);
```

High impedance

- 'Z' value of std_logic
  - high impedance or an open circuit
  - not a normal logic value
  - only synthesizable by a tri-state buffer



oe	y
0	Z
1	a_in

  - the code of the tri-state buffer is

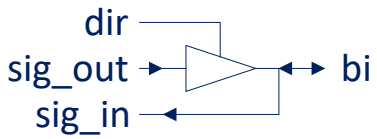
```
y <= a_in when oe='1' else 'Z';
```

Note: In the majority of FPGA devices, tri-state buffers only exist in the I/O blocks (IOB). If they are required internally they can be replaced by multiplexers.

Bidirectional port

- The most common application for a tri-state buffer is to implement a bidirectional port
  - e.g.

```
entity bi_demo is
  port (
        bi: inout std_logic;
        ...
begin
sig_out <= output_expression;
...
some_signal <= expression_with_sig_in;
...
bi <= sig_out when dir = '1' else 'Z';
...
sig_in <= bi;
...
```



- the `dir` signal controls the direction of signal flow of the `bi` pin
  - if `dir` = '0' the pin is used as an input
  - if `dir` = '1' the pin is used as an output

Review of some good practices

- To build synthesizable circuits only a small set of data types and operators are needed
  - Use `std_logic` and `std_logic_vector` data types in entity port declaration + internal signals that involve no arithmetic operations
  - Use the IEEE `numeric_std` package + `unsigned` or `signed` data types for the internal signals that involve arithmetic operation
  - Use the data type conversion functions to convert signals and expressions among the `std_logic_vector` and various numerical data types
  - Use the '`Z`' value only to infer a tri-state buffer

Routing circuit with concurrent assignment statements

- *Conditional signal assignment*
- *Selected signal assignment*

} concurrent statements

  - similar to *if* and *case* in conventional programming language
  - instead of being executed sequentially, these statements are mapped to a routing network during synthesis
- *Conditional signal assignment*

```
signal_name <= value_expr_1 when boolean_expr_1 else
               value_expr_2 when boolean_expr_2 else
               ...
               value_expr_n;
```

Conditional signal assignment

- the Boolean expressions are evaluated successively
- when one is found to be true the corresponding value expression is assigned to the signal
- the `value_expr_n` is assigned if all earlier expressions are evaluated to be false

- Example with an *n*-to-2^n binary decoder
  - it asserts 1 bit of the 2^n-bit output according to the input combination

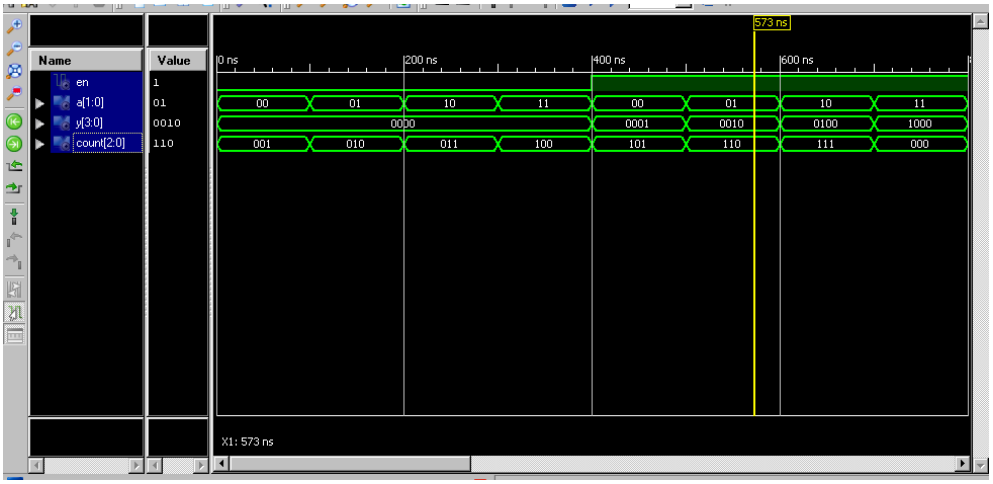
n-to-2^n binary decoder with enable

```
library ieee;
use ieee.std_logic_1164.all;
entity decoder_2_to_4 is
   port(
        a: in std_logic_vector (1 downto 0);
        en: in std_logic;
        y: out std_logic_vector (3 downto 0)
   );
end decode_2_to_4;
architecture cond_arch of decoder_2_to_4 is
begin
   y <= "0000" when (en='0') else
        "0001" when (a="00") else
        "0010" when (a="01") else
        "0100" when (a="10") else
        "1000";                    -- a="11"
end cond_arch;
```

	input		output
en	a(1)	a(0)	y
0	-	-	0000
1	0	0	0001
1	0	1	0010
1	1	0	0100
1	1	1	1000

Ex. 5

Logic simulation results of the conditional signal assignment



Selected signal assignment

- *Selected signal assignment*

```
with   sel select
       sig <=   value_expr_1 when choice_1,
                value_expr_2 when choice_2,
                ...
                value_expr_n when others;
```

the reserved word, **others**, is used in the end to cover unused values

  - similar to *case* in a traditional programming language
  - assigns an expression to a signal according to the value of the *sel* signal
  - choices have to be *mutually exclusive* (i.e., no value can be used more than once) + *all inclusive* (i.e., all values must be used) ⟶ all possible values of `sel` must be covered by one and only one choice

n-to-2^n binary decoder with enable

```
architecture sel_arch of decoder_2_to_4 is
    signal sel: std_logic_vector (2 downto 0);
begin
    sel <= en & a;
    with sel select
        y <= "0000" when "000"|"001"|"010"|"011", -- en='0'
             "0001" when "100",
             "0010" when "101",
             "0100" when "110",
             "1000" when others;                  -- sel="111"
end sel_arch;
```

the symbol | is used when the choice is more than one value

Ex. 6

- en and a signals are concatenated to form 3-bit signal sel → selection signal
- the code exhaustively lists all possible combinations of the sel signal and the corresponding output values

Routing circuit with if and case statements

- If statement

```
if boolean_expr_1 then
    sequential_statements;
elsif boolean_expr_2 then
    sequential_statements;
elsif boolean_expr_3 then
    sequential_statements;
...
else
    sequential_statements;
end if;
```

If and case statements are intended exclusively for sequential code ⇒ they **can only be used inside a process**

- the Boolean expressions are evaluated sequentially until an expression is evaluated as *true* or the else branch is reached
- the statements in the corresponding branch are executed

n-to-2^n binary decoder with enable

```
architecture if_arch of decoder_2_to_4 is
begin
    process (en, a)
        begin
            if (en='0') then
                y <= "0000";
            elsif (a="00") then
                y <= "0001";
            elsif (a="01") then
                y <= "0010";
            elsif (a="10") then
                y <= "0100";
            else
                y <= "1000";          -- a="11"
            end if;
    end process;
end if_arch;
```

Ex. 7

The `case` statement

- Uses signal `sel` to select a set of sequential statements for execution

```
case sel is
      when choice_1 =>
               sequential statements;
      when choice_2 =>
               sequential statements;
      ...
      when others =>
               sequential statements;
end case;
```

Remember:
exclusively for sequential code

the **`others`** term at the end covers any unused value

  - `choice_i` must be a valid value or a set of valid values of `sel`
  - choices have to be *mutually exclusive* and *all inclusive*

n-to-2^n binary decoder with enable

```
architecture case_arch of decoder_2_4 is
     signal sel: std_logic_vector (2 downto 0);
begin
     sel <= en & a; -- en and a signals concatenated to form the sel signal
     process (sel)
     begin
          case sel is
               when "000"|"001"|"010"|"011" =>
                 Y <= "0000";
               when "100" =>
                 Y <= "0001";
               when "101" =>
                 Y <= "0010";
               when "110" =>
                 Y <= "0100";
               when others =>
                 y <= "1000";                -- sel="111"
          end case;
     end process;
end case_arch;
```

Ex. 8

`case` versus `if`

- After optimization, the general tendency is for a circuit synthesized from a VHDL code based on `if` not to differ from that based on `case`
  - e.g. these codes implement the same physical multiplexer circuit

```
---- With if
if (sel="00")          then x<=a;
     elsif (sel="01") then x<=b;
     elsif (sel="10") then x<=c;
     else x<=d;
end if;
```

```
---- With case
case sel is
     when "00"   => x<=a;
     when "01"   => x<=b;
     when "10"   => x<=c;
     when others => x<=d;
end case;
```

Comparison to concurrent statements

- The `if` and `case` statements ⇔ to the conditional and selected signal assignment statements
- However, `if` or `case` statements allow any number and any type of sequential statements in their branches ⇒ more flexible and versatile
- Disciplined use can make the code more descriptive and even make a circuit more efficient

Using sequential code to design combinational circuits

- Sequential code can be used to implement either
  - sequential circuits ⇒ registers are necessary ⇒ they are inferred by the compiler,
    or
  - combinational circuits ⇒ **no registers at all**
- If the code is intended for a combinational circuit, then the complete truth-table should be clearly specified in the code
- In order to satisfy the criteria above, the following rules should be observed:

Using sequential code to design combinational circuits

- Rule 1: Make sure that **all input signals** used (read) in the PROCESS appear in its sensitivity list
- Rule 2: Make sure that **all combinations** of the input/output signals are included in the code; that is, make sure that, by looking at the code, the circuit's complete truth-table can be obtained (indeed, this is true for both sequential as well as concurrent code)

Using sequential code to design combinational circuits

- Failing to comply with rule 1 → the synthesizer simply issues a warning saying that a given input signal is not included in the sensitivity list, and then proceed as if the signal were included
  - even though no damage is caused to the design in this case, it is a good design practice to always take rule 1 into consideration
- Failing to comply with rule 2 → incomplete specifications of the output signals might cause the synthesizer to infer latches in order to hold their previous values

WARNING: [Synth 8-327] inferring latch for variable <signal name>

Using sequential code to design combinational circuits

- Fai…es a w…the se…led
  - …is a
- Fa…s of the output signals…t cause the synthesizer to infer latches in order to hold their previous values

- The VHDL standard specifies that a signal keeps its previous value if it is not assigned in a process
- During synthesis, this infers an internal state (via a closed feedback loop) or a memory element (such as a latch)

WARNING: [Synth 8-327] inferring latch for variable <signal name>

WARNING: [Synth 8-327] inferring latch for variable <signal name>

- If the logic is intended to be combinatorial logic ⇒ no latches
- Latches break the synchronous path
- Look through the "Start Writing Synthesis Report" section of the "Vivado Synthesis Report" to see if there are latches inferred
  - check this by looking at the "Report Cell Usage" section of the Synthesis report

```
Report Cell Usage:
+------+-------+------+
|      |Cell   |Count |
+------+-------+------+
|1     |BUFG   |     2|
|2     |CARRY4 |    12|
|3     |LUT1   |    49|
|4     |LUT2   |    12|
|5     |LUT3   |    31|
|6     |LUT4   |    18|
|7     |LUT5   |     8|
|8     |LUT6   |    50|
|9     |FDRE   |    86|
|10    |LDC    |     1|
|11    |IBUF   |    19|
|12    |OBUF   |    36|
+------+-------+------+
```

WARNING: [Synth 8-327] inferring latch for variable <signal name>

- If inferred latches is not expected, there are several things to check:
  - ensure if-else statements assign the net to a value under all conditions
  - ensure case statements assign a value to all signals in all conditions
  - if you are intending to infer a register, make sure to include the system clock in the process

from: AR# 39136
Design Assistant for XST – Reasons why a latch is inferred
Xilinx Support

Inferred latch example

```
process (a,b)
begin
   if (a > b)            -- then eq (equal) not assigned
       gt <= '1';        -- in this branch
   elsif (a = b) then    -- gt (greater than) not assigned
       eq <= '1';        -- in this branch
   end if;               -- else branch is omitted
end process;
```

- `gt` keeps its previous value when the `a > b` expression is false and a latch is inferred accordingly

The correct code to avoid inferred latch

```
process (a,b)
begin
   if (a > b) then
        gt <= '1';
        eq <= '0';
   elsif (a = b) then
        gt <= '0';
        eq <= '1';
   else
        gt <= '0';
        eq <= '0';
   end if;
end process;
```

All if-else statements assign `gt` and `eq` to a value under all conditions

The correct code to avoid inferred latch

```
process (a,b)
begin
   if (a > b) then
        gt <= '1';
        eq <= '0';
   elsif (a = b) then
        gt <= '0';
        eq <= '1';
   else
        gt <= '0';
        eq <= '0';
   end if;
end process;
```

Since multiple sequential signal assignment statements are allowed inside a process, the problem can also be corrected by assigning a default value in the beginning

```
process (a,b)
begin
  gt <= '0';
  eq <= '0';
  if (a > b) then
     gt <= '1';
  elsif (a = b) then
     eq <= '1';
  end if;
end process;
```

The correct code to avoid inferred latch

```
process (a,b)
begin
   if (a > b) then
        gt <= '1';
        eq <= '0';
   elsif (a = b) then
        gt <= '0';
        eq <= '1';
   else
        gt <= '0';
        eq <= '0';
   end if;
end process;
```

Since multiple sequential signal assignment statements are allowed inside a process, the problem can also be corrected by assigning a default value in the beginning

```
process (a,b)
begin
   gt <= '0';
   eq <= '0';
   if (a > b) then
      gt <= '1';
   elsif (a = b) then
      eq <= '1';
   end if;
end process;
```


Edge Sensitive Elements

`(clk'event and clk='1')` vs `rising_edge(clk)`

- Two ways to describe edge sensitive elements

  (assuming that signal `clk` is of type std_logic):

  - `clk'event and clk = '1'` returns true if signal `clk` has a transition and its new value is '1'
  - If the previous value of `clk` is `'0'`, then this expression correctly detects a rising edge ⇒ no problem when describing hardware

(clk'event and clk='1') vs rising_edge(clk)

- If performing simulation
  - Any transition that ends at '1' is considered as a rising edge
  - Hence, changes from 'H' (weak '1') to '1' return true while transitions from '0' to 'H' return false
  - As a result, clk'event and clk = '1' is only safe if clk toggles between '0' and '1'

(clk'event and clk='1') vs rising_edge(clk)

- rising_edge also returns true for transitions from '0' to 'H' and returns false for transitions from 'H' to '1' (as well as changes from 'X' to '1')
- For synthesis ⇒ no difference between both alternatives as the synthesis tool assumes that transitions on clock signals are "clean" (i.e., clock signals toggle between '0' and '1' only)
- However, for simulation it is safer to use rising_edge

(clk'event and clk='0') vs falling_edge(clk)

- To detect a falling edge use `falling_edge`
- If performing simulation
  - Any transition that ends at `'0'` is considered as a falling edge
  - Hence, changes from 'L' (weak '0') to `'0'` return true while transitions from `'1'` to 'L' return false
- `rising_edge (clk)` and `falling_edge(clk)` are defined in package std_logic_1164
  - it returns true if signal "clk" has a rising or falling transition, respectively

Example of an 8-bit register with synchronous reset

```
PROCESS (clk)
BEGIN

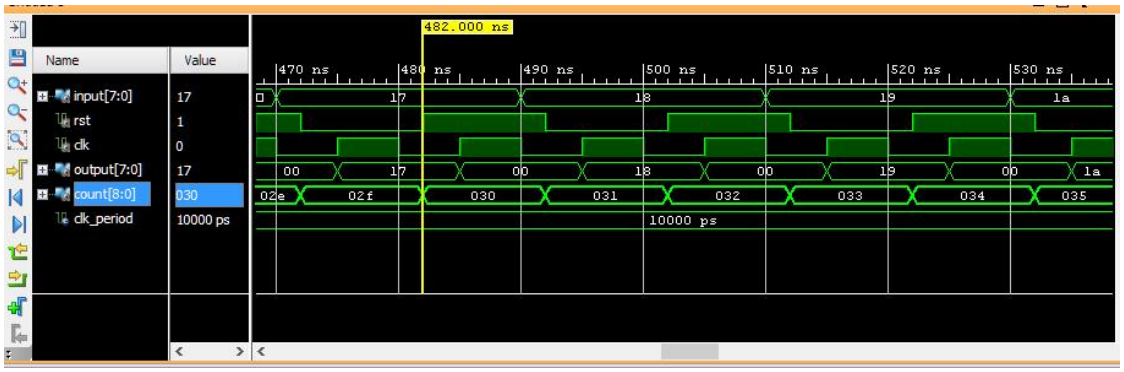
  IF rising_edge(clk)
   THEN
         IF (rst='1') THEN
               output <= "00000000";
         ELSE
               output <= input;
         END IF;
  END IF;

END PROCESS;
```

The reset signal must always be present when specifying synchronous memory elements → needed to specify the initial value

Ex. 9

Logic simulation results



Example of an 8-bit register with asynchronous reset

```
PROCESS (clk, rst)
BEGIN
    IF (rst='1') THEN
          output <= "00000000";
      ELSIF rising_edge(clk) THEN
            output <= input;

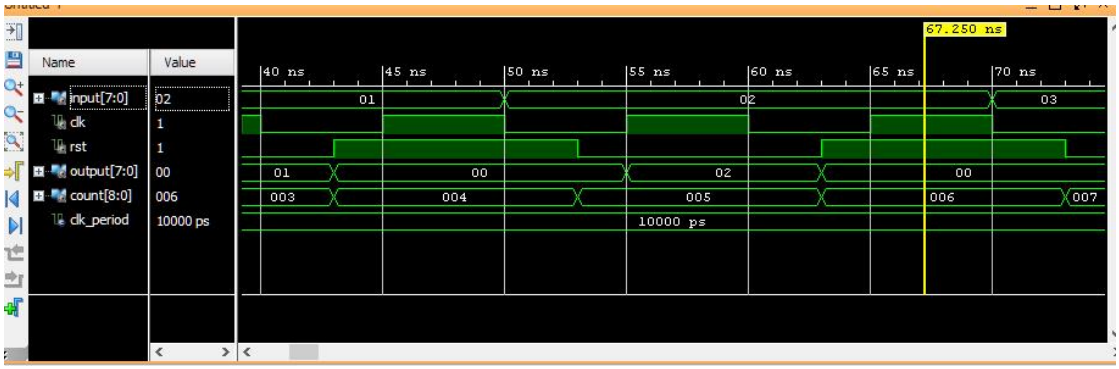
    END IF;

END PROCESS;
```

Again, the reset signal, be it synchronous or asynchronous, must be present when specifying memory elements

Ex. 10

Logic simulation results

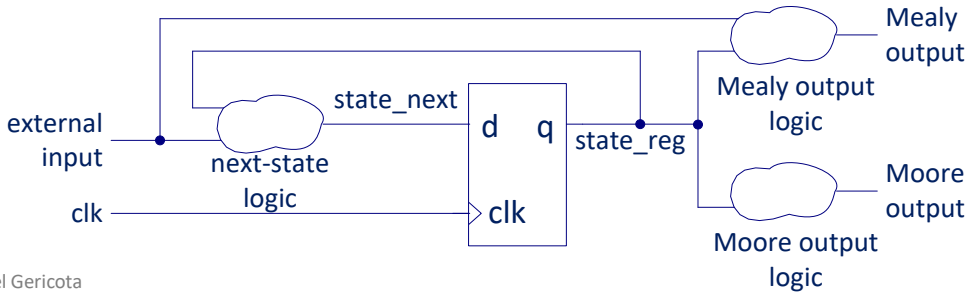


FSM (finite state machine)

- Used to model a system that transits among a finite number of internal states
  - transitions depend on the current state and external input
  - unlike a regular sequential circuit (like counters or shifters), state transitions do not exhibit a simple, repetitive pattern
  - next-state logic → constructed from scratch → sometimes known as "random" logic
  - different from the next-state logic of a regular sequential circuit → composed mostly of "structured" components – increment functions and shifters

Moore and Mealy FSM

- An FSM may be of two types
  - *Moore machine* → the output depends only on state
  - *Mealy machine* → the output depends on state and external input(s)
- Both types may coexist in a complex FSM

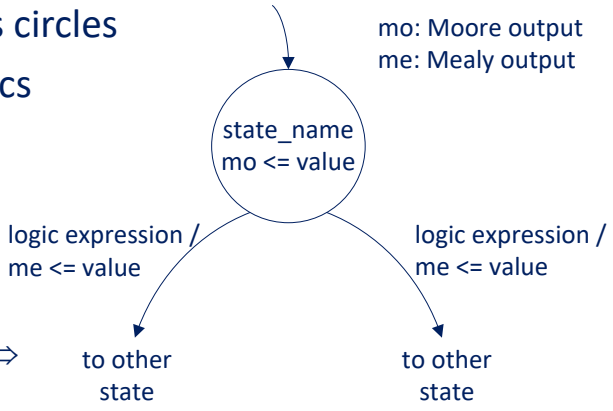


FSM representation

- FSM ⇒ specified by an *abstract state diagram* or *ASM chart* (algorithmic state machine chart)
  - both capture the FSM's input, output, states, and transitions in a graphical representation
  - both provide the same information
  - abstract state diagram → more compact, better for simple applications
  - ASM chart → resembles a flowchart → more descriptive for applications with complex transition conditions and actions

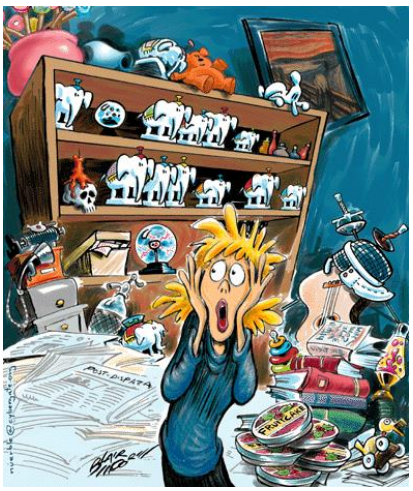
Abstract state diagram

- Composed of:
  - nodes ⟶ states drawn as circles
  - annotated transitional arcs
    - logic expression = *f*(input signals) is associated with each transition arc ⟶ it represents a specific condition
    - if logic expression = true ⟹ the arc is taken



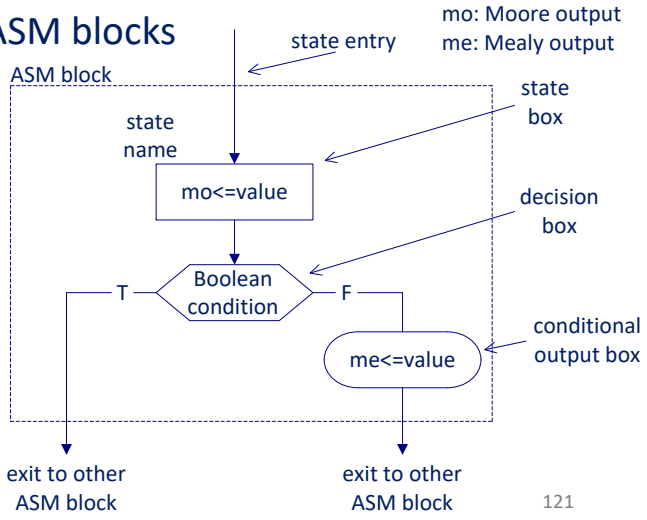
Abstract state diagram

- Moore output values ⟶ placed inside the circle ⟶ depend only on the state
- Mealy output values ⟶ associated with the logic expressions of transition arcs ⟶ depend on current state and external input(s)
- To reduce clutter in the diagram ⟶ only asserted output values are listed
- Output signals take the default (i.e., not asserted) value otherwise



Algorithmic state machine (ASM) chart

- Composed of a network of ASM blocks
  - an ASM block consists of:
    - one state box ⇒ a state in an abstract state diagram
    - an optional network of decision boxes + conditional output boxes
  - asserted Moore output values are listed inside the box (notice that it has only one exit path)

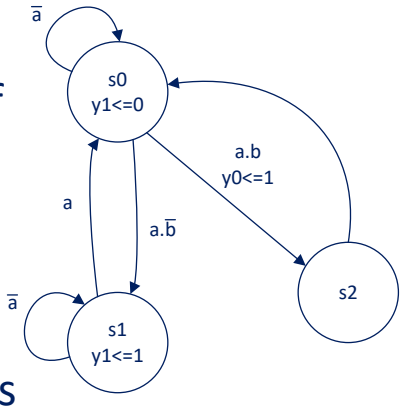


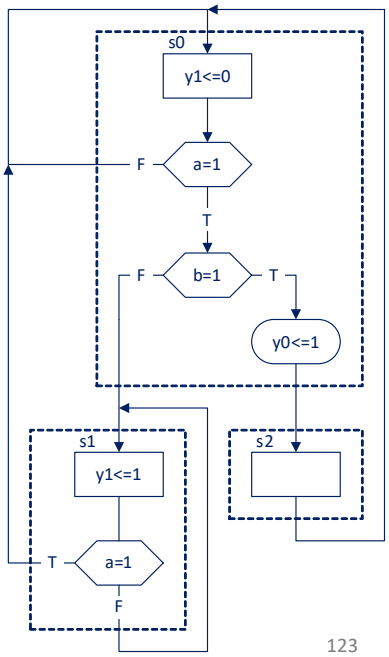
Algorithmic state machine (ASM) chart

- decision box ⟶ tests the input condition and determines which exit path to take
  - two exit paths ⟶ T (`true`) and F (`false`) ⇒ Boolean values of the condition
- conditional output boxes ⟶ list asserted Mealy output values ⟶ usually placed after a decision box
- indicates that the listed output signal can be activated only when the corresponding condition in the decision box is met (`true`)

Abstract state diagram vs ASM

- An example of the same FSM represented using the two graphical methodologies



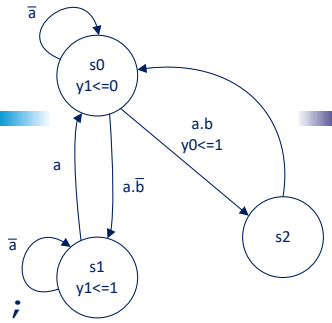


FSM code development

1. separate the state registers
2. derive the code for the combinational next-state logic and output logic

- The code for the next-state logic follows the flow of the state diagram
- VHDL's *enumerated data type* is used to represent the FSM's states for clarity and flexibility

State assignment



- For the previous example:
  - create a user-defined enumerated data type for states s0, s1, and s2

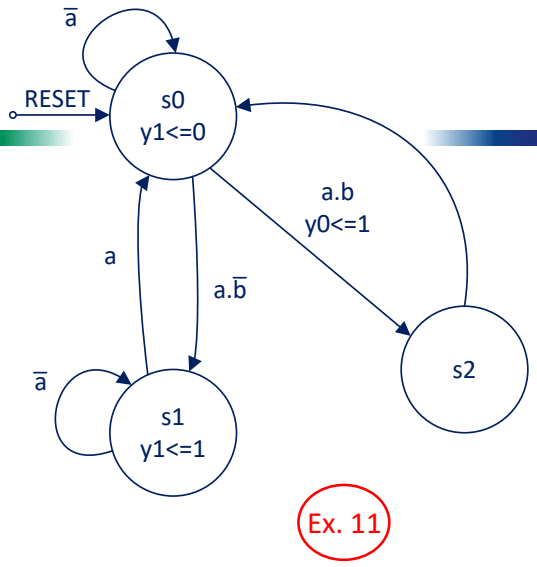
```
type eg_state_type is (s0, s1, s2);
```

  - the data type simply lists (i.e., *enumerates*) all symbolic values
  - once defined, it can be used for the signals, as in

```
signal state_reg, state_next: eg_state_type;
```

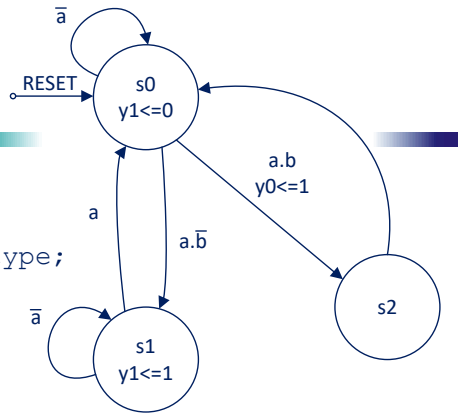
  - during synthesis, the tool automatically maps the values in an enumerated data type to binary representations → a process known as *state assignment*

FSM example




```
library ieee;
use ieee. std_logic_1164. all;
entity fsm_eg is
   port (
        clk: in std_logic;
        reset: in std_logic;
        a: in std_logic;
        b: in std_logic;
        y0: out std_logic;
        y1: out std_logic
        );
end fsm_eg;
```

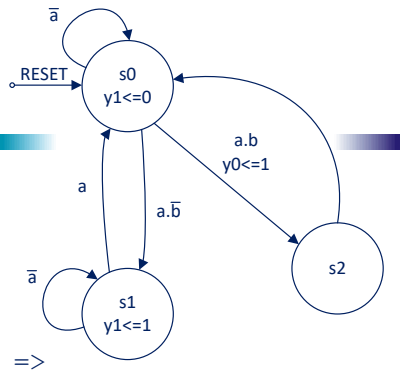
FSM example




```
architecture mult_seg_arch of fsm_eg is
   type eg_state_type is (s0, s1, s2);
   signal state_reg, state_next: eg_state_type;
begin   -- state register
   process(clk, reset)
   begin
        if (reset='1') then
               state_reg <= s0;
        elsif (clk'event and clk='1') then
               state_reg <= state_next;
        end if;
   end process;
```

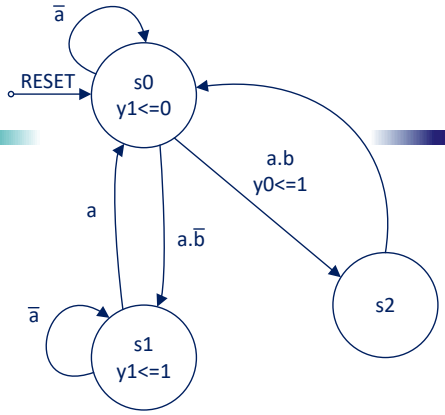
The reset signal is always needed to specify the initial state

FSM example




```
-- next-state logic
process(state_reg, a, b)
begin
   case state_reg is
      when s0 =>
         if a='1' then
            if b='1' then
               state_next <= s2;
            else
               state_next <= s1;
            end if;
         else
            state_next <= s0;
         end if;
      when s1 =>
         if (a='1') then
            state_next <= s0;
         else
            state_next <= s1;
         end if;
      when s2 =>
         state_next <= s0;
      end case;
end process;
```

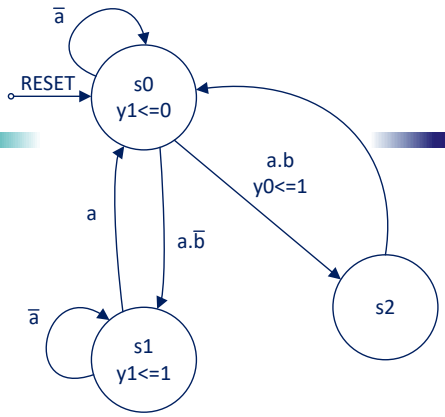
FSM example




```
-- Moore output logic

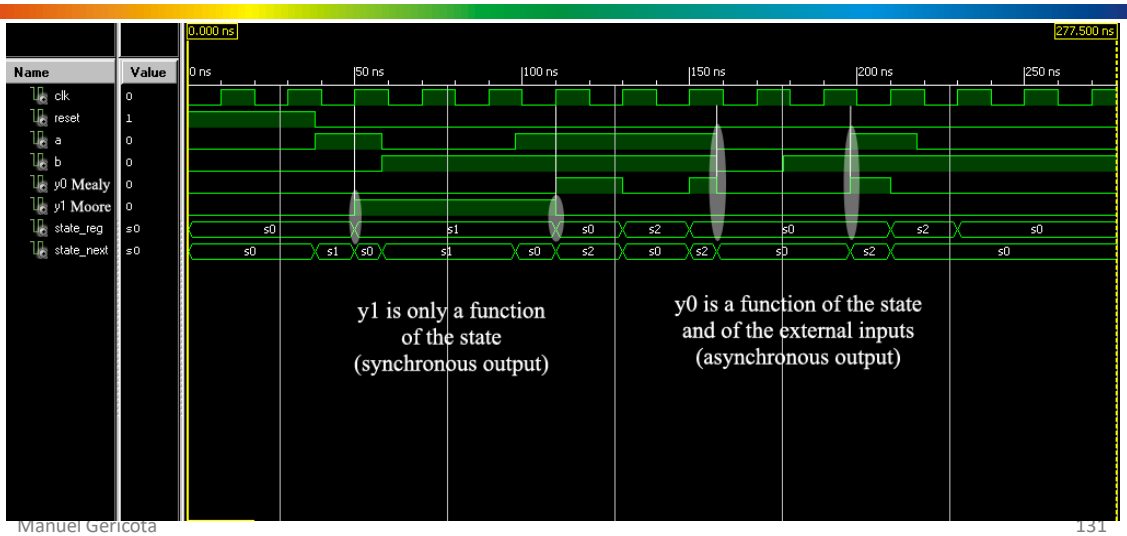
   process(state_reg)
   begin
      case state_reg is
         when s0 | s2 =>
            y1 <= '0';
         when s1 =>
            y1 <= '1' ;
      end case;
   end process;
```


FSM example




```
-- Mealy output logic
   process(state_reg,a,b)
   begin
      case state_reg is
         when s0 =>
            if (a='1') and (b='1') then
               y0 <= '1' ;
            else
               y0 <= '0' ;
            end if;
         when s1 | s2 =>
            y0 <= '0' ;
      end case;
   end process;
end mult_seg_arch;
```

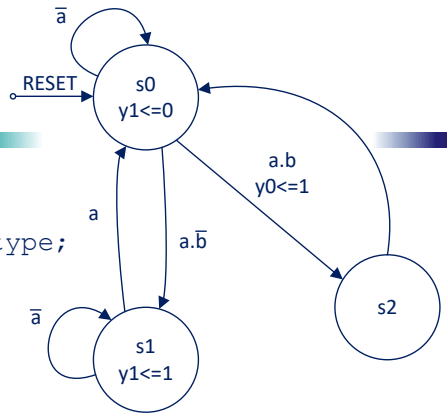
FSM logic simulation results



131

FSM example with merged combinational logic

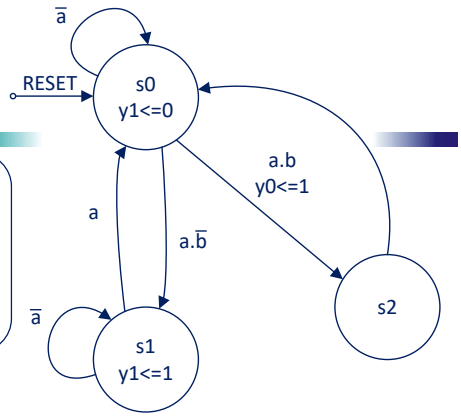
```
architecture two_seg_arch of fsm_eg is
   type eg_state_type is (s0, s1, s2);
   signal state_reg, state_next: eg_state_type;
begin
-- state register
   process(clk,reset)
   begin
      if (reset='1') then
         state_reg <= s0;
      elsif (clk'event and clk='1') then
         state_reg <= state_next;
      end if;
end process;
```



Ex. 12

132

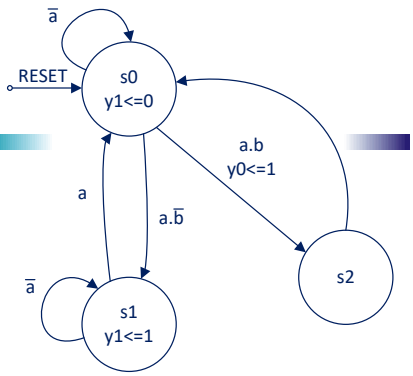
FSM example with merged combinational logic



Definition of default state and output values

```
-- next-state/output logic
   process (state_reg, a, b)
   begin
      state_next <= state_reg; -- by default keeps in the same state
      y0 <= '0';               -- default 0
      y1 <= '0';               -- default 0
```


FSM example with merged combinational logic



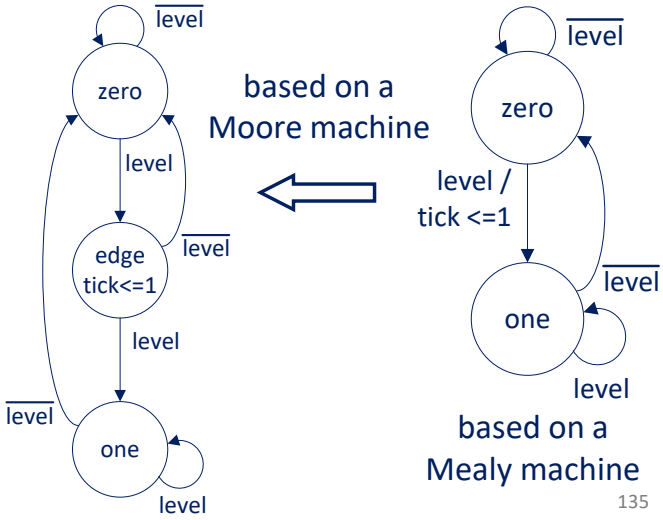

```
case state_reg is
   when s0 =>
      if a='1' then
         if b='1' then
            state_next <= s2;
            y0 <= '1';
         else
            state_next <= s1;
         end if;
      end if;
   when s1 =>
      y1 <= '1';
      if (a='1') then
         state_next <= s0;
      end if;
```

The code for the next-state logic and output logic closely follows the state diagram

```
      when s2 =>
         state_next <= s0;
   end case;
end process;
end two_seg_arch;
```

Comparing Moore and Mealy machines – an example

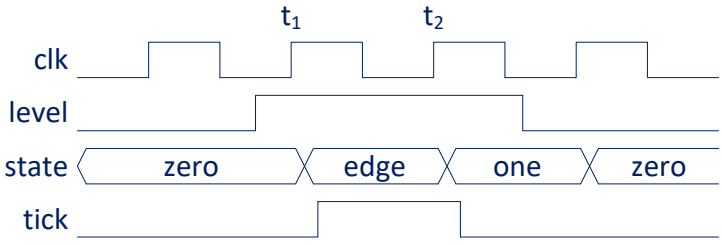
- Rising-edge detector
  - rising-edge detector → a circuit that generates a short, one-clock-cycle pulse (called a *tick)* when the input signal changes from '0' to '1'
  - usually used to indicate the beginning of a slow time-varying input signal



The Moore-based design

- The 'zero' and 'one' states indicate the input signal has been '0' and '1' for awhile
- Input changes to '1' in the 'zero' state ⇒ rising edge occurs
- FSM moves to the 'edge' state → the output 'tick' is asserted

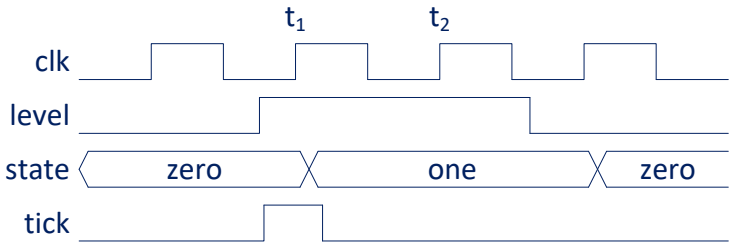
Timing diagram of the Moore-based design



The Mealy-based design

- The ‘zero’ and ‘one’ states have similar meaning
- If, while in the ‘zero’ state, the input changes to ‘1’ ⇒ the output is asserted immediately → tick <=‘1’
- FSM moves to ‘one’ state at the rising edge of the next clock ⇒ the output is deasserted

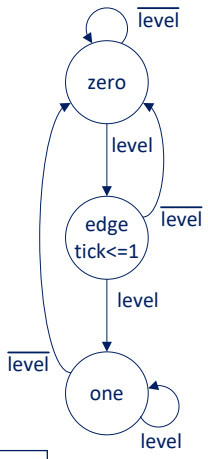
Timing diagram of the Mealy-based design



Code for the Moore-based design

```
library ieee;
use ieee. std_logic_1164. all;
entity edge_detect is
  port(
      clk, reset: in std_logic;
      level: in std_logic;
      tick: out std_logic
      );
end edge_detect;
```

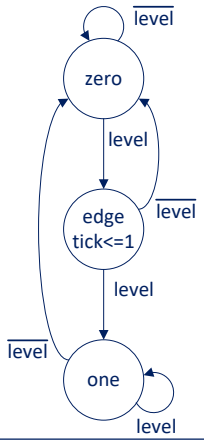
Inputs and output of the rising-edge detector



Ex. 13

Code for the Moore-based design

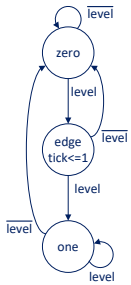
```
architecture moore_arch of edge_detect is
   type state_type is (zero, edge, one);
   signal state_reg, state_next : state_type;
begin -- state register
   process (clk, reset)
   begin
        if (reset='1') then
               state_reg <= zero;
        elsif (clk'event and clk='1') then
               state_reg <= state_next;
        end if;
   end process;
```



State transitions are synchronous ⇒ output variation also synchronous (Moore machine)

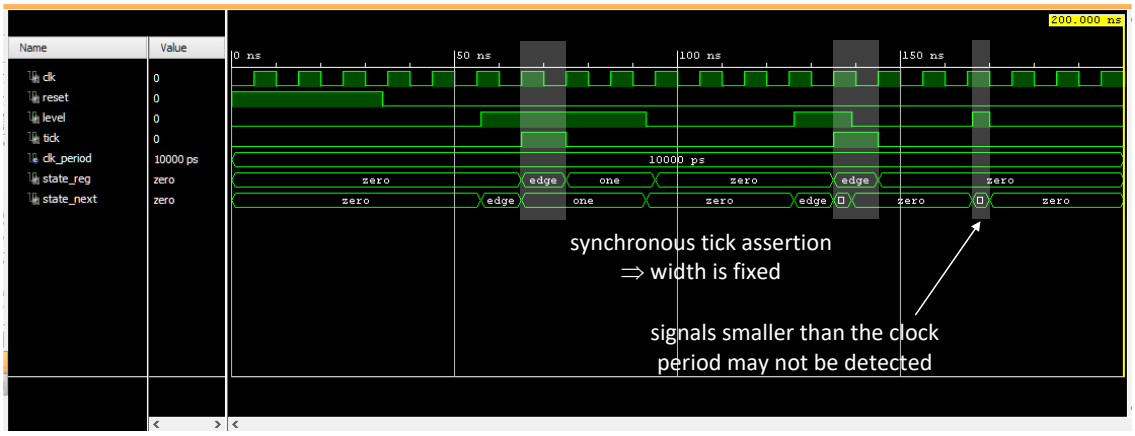
Code for the Moore-based design

```
process (state_reg,level)  -- next-state / output logic
begin
   state_next <= state_reg;
   tick <= '0';              -- by default tick is zero
      case state_reg is
         when zero=>
            if level= '1' then
               state_next <= edge;
            end if;
         when edge =>
            tick <= '1';
            if level= '1' then
               state_next <= one;
            else
               state_next <= zero;
            end if;
         when one =>
            if level= '0' then
               state_next <= zero;
            end if;
      end case;
end process;
end moore_arch;
```



'tick' changes are synchronous ⇐ change only when state changes

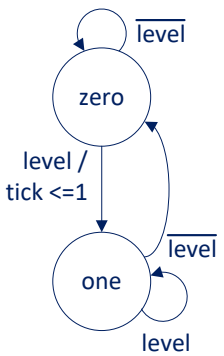
Moore-based design logic simulation results



Code for the Mealy-based design

```
architecture mealy_arch of edge_detect is
   type state_type is (zero, one);
   signal state_reg, state_next: state_type;
begin
-- state register
   process(clk,reset)
   begin
      if (reset='1') then
         state_reg <= zero;
      elsif (clk'event and clk='1') then
         state_reg <= state_next;
      end if;
   end process;
```

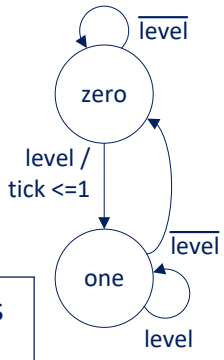
one less state declaration
(the 'edge' state)



Ex. 14

Code for the Mealy-based design

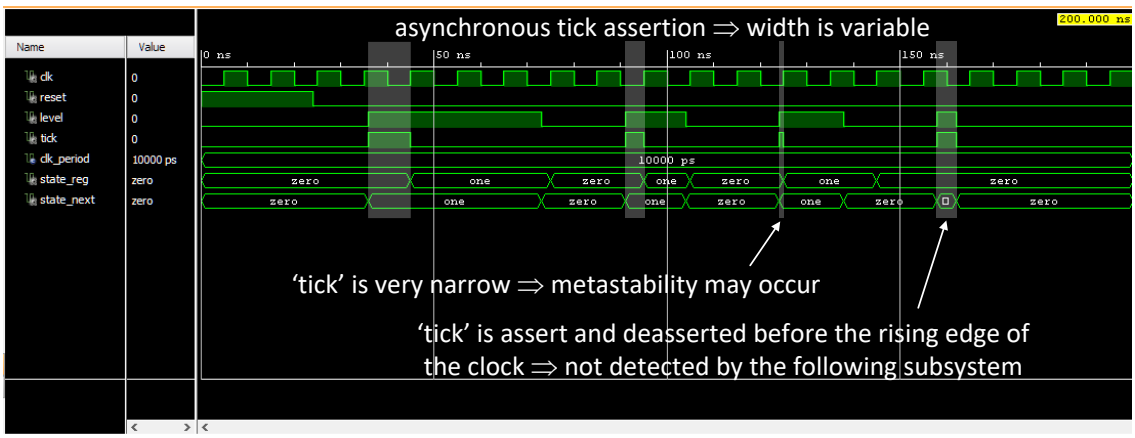
```
   process (state_reg, level)    -- next-state/output logic
   begin
      state_next <= state_reg;
         tick <= '0';            -- by default tick is zero
         case state_reg is
            when zero=>
               if level= '1' then
                  state_next <= one;
                  tick <= '1';
               end if;
            when one =>
               if level= '0' then
                  state_next <= zero;
               end if;
         end case;
   end process;
end mealy_arch;
```



'tick' assertion is asynchronous

143

Mealy-based design logic simulation results



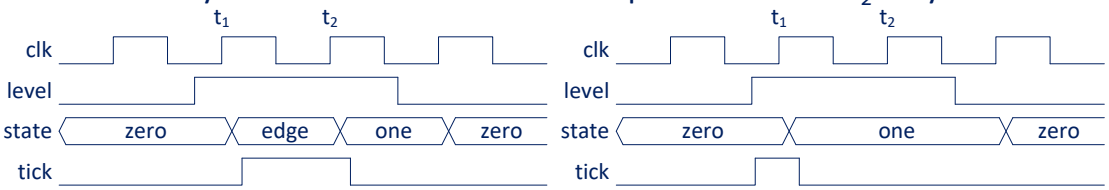
144

Comparison between the two implementations

- Both Moore machine- and Mealy machine-based designs can generate a tick at the rising edge of the input signal, but:
  - the Mealy machine-based design requires fewer states and responds faster
  - however, the width of its output may vary ⇒ input glitches may be passed to the output
- The choice between the two designs depends on the subsystem that uses the output signal
  - most of the time the subsystem is a synchronous system that shares the same clock signal

Comparison between the two implementations

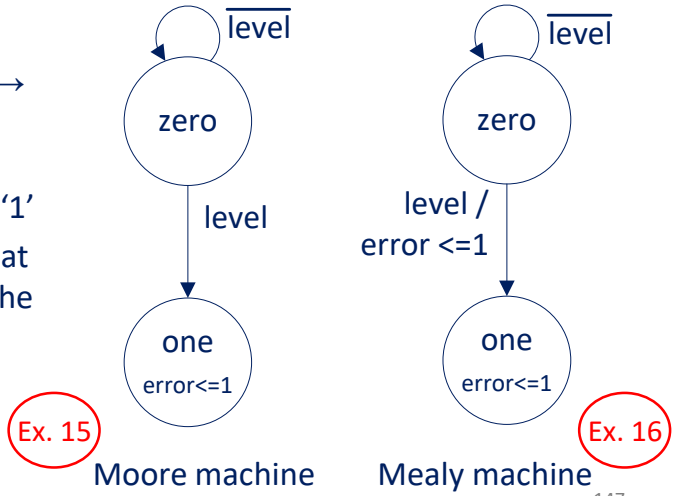
- FSM's output is sampled only at the rising edge of the clock ⇒ the width and glitches do not matter as long as the output signal is stable around the edge → no guarantees
- note that the Mealy output signal is available for sampling at t_1, one clock cycle faster than the Moore output available at t_2 only



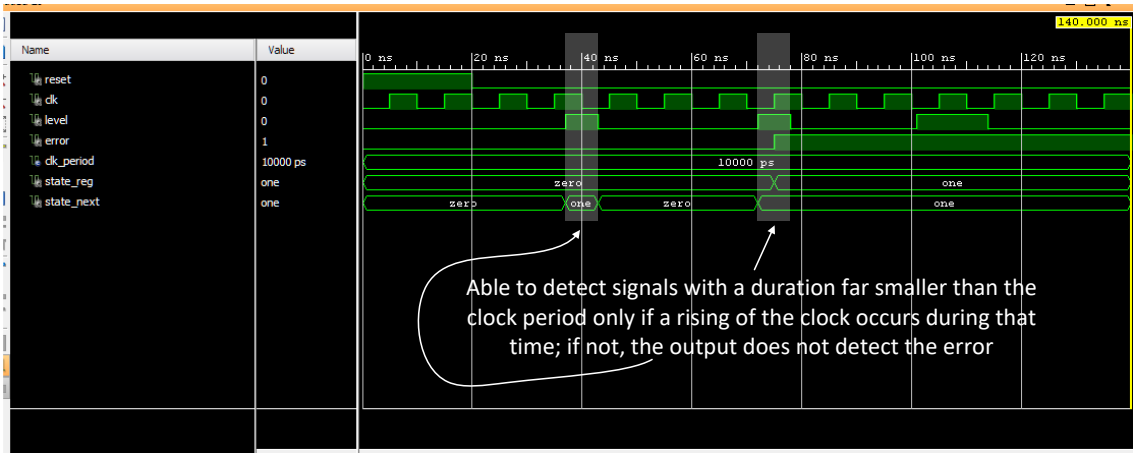
⇒ the Mealy machine-based circuit may be preferred for this type of application, **but metastability may occur!**

Another example – an error detector

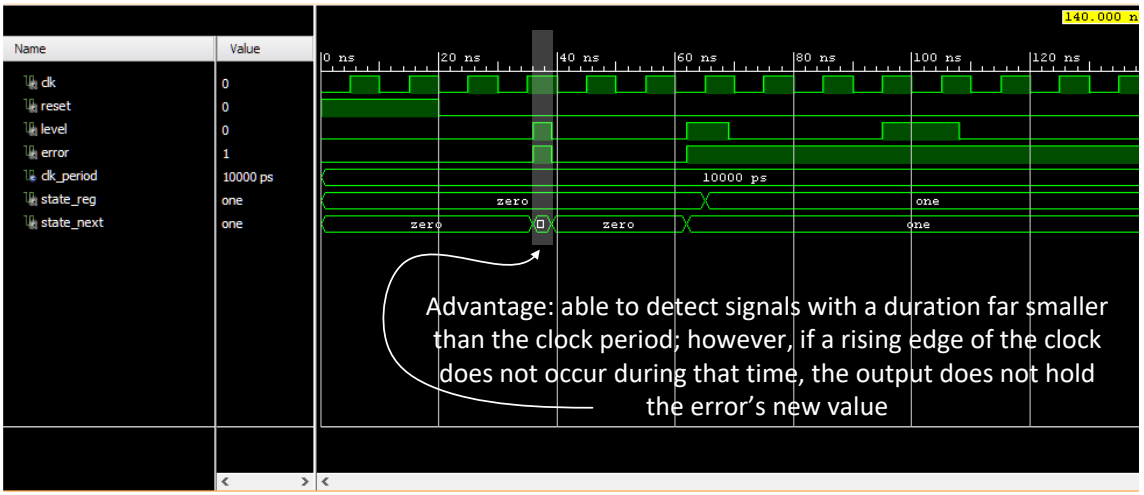
- Error detector
  - the error detector circuit → changes permanently the output when the input signal changes from '0' to '1'
  - usually used to indicate that an error occurred during the operation of the circuit, despite being it or not a permanent error



Moore-based design logic simulation results



Mealy-based design logic simulation results



The `wait` statement

- `wait` operation sometimes ⇔ to `if`
- Usually used to write test benches
- `wait` statements → exclusively for sequential code ⇒ it can only be used inside a process
- Syntax:

  `wait until` `signal_condition;` → it accepts only one signal

  `wait on` `signal1 [, signal2, ...];` → it accepts multiple signals

  `wait for` `time;`

The `wait` statement

- **IMPORTANT: when using `wait` ⇒ `process` cannot has a sensitivity list** ⇒ `wait until` must be the first statement in the `process`
  - the `process` is executed every time the condition is met
- `wait on` accepts multiple signals
  - The `process` is put on hold until any of the signals listed changes

`wait until`

- e.g. 8-bit register with synchronous reset

Ex. 17

```
PROCESS     -- no sensitivity list
BEGIN
 WAIT UNTIL (clk'EVENT AND clk='1');
      IF (rst='1') THEN
           output <= "00000000";
      ELSE
           output <= input;
      END IF;
END PROCESS;
```

Remember: the reset signal must always be present when specifying synchronous memory elements → needed to specify the initial value

wait on

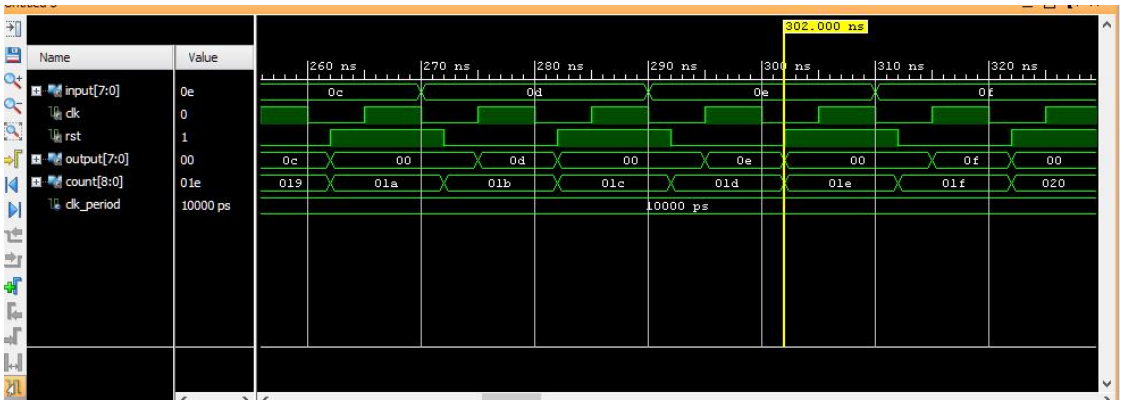
- e.g. 8-bit register with asynchronous reset

```
PROCESS
BEGIN
  WAIT ON clk, rst;
  IF (rst='1') THEN
    output <= "00000000";
  ELSIF (clk'event AND clk='1') THEN
    output <= input;
  END IF;
END PROCESS;
```

multiple signals ⇒ the `process` is put on hold until one of the signals change

Ex. 18

Logic simulation results



`wait for`

- `wait for` is intended for simulation only
  - waveform generation for test benches
  - e.g. `wait for 5 ns;`

time is a physical dimension ⇒ it can not be synthesized

Signals and variables

- Non-static data values
  - `signal` and `variable`
- Static (default) values
  - `constant` and `generic`
- `constant` and `signal`
  - global (seen by the whole code) either concurrent or sequential
- `variable`
  - local ⇒ it can only be used inside a `process` (⇒ only with sequential code)
  - its value can never be passed out directly

signal

- Serves to pass values in and out of the circuit and among its internal units
  - a signal represents circuit interconnection (wires)
  - a signal may represent a register when used in a sequential description, e.g., `state register` in a FSM
  - all ports of an `entity` are signals by default
- Syntax:

```
SIGNAL name : type [range] [:= initial_value];
```

- A signal can be declared in a `package`, `entity`, or `architecture`

The initial value is not synthesizable; it is only considered in simulation

signal

- When declared in a `package`
  - truly global ⇐ the `package` can be used by several entities
- When declared in an `entity` (after `port`)
  - it is global to all architectures that follow that `entity`
- When declared inside an `architecture` (in its declarative part)
  - it is global only to that architecture's code
- Signals can not be declared in the declarative part of a `process`

signal

- Important notice
  - when used inside a section of sequential code, the signal update is not immediate
  - it assumes its new value only when the corresponding `process` is concluded
- Assignment operator for a signal → '<='
  - e.g. `count <= 35;`
- Remember
  - when a signal is assigned multiple times inside a `process`, only the last assignment takes effect

variable

- A `variable` represents only local information
- It can only be used inside a `process` – in sequential code
  - can be only declared in the declarative part of a `process` not in an `architecture`
  - its value cannot be passed out directly
- Its update is immediate ⇒ the new value can be promptly evaluated or used in the next line of code
- Syntax:

```
VARIABLE name : type [range] [:= init_value];
```

The initial value is not synthesizable; it is only considered in simulation

variable

- Assignment operator for a variable → ‘ := ’
- e.g.

```
VARIABLE control: STD_LOGIC := ‘0’;
VARIABLE count: INTEGER RANGE 0 TO 100;
count := 35;
VARIABLE y: STD_LOGIC_VECTOR (7 DOWNTO 0) := “10001000”;
```


signal ↔ variable

- Signals can be assigned to variables and variables can be assigned to signals
- e.g.

```
signal s_single : std_logic;
signal s_vector : std_logic_vector(0 to 3);
variable v_single : std_logic;
variable v_vector : std_logic_vector(0 to 3);
s_single <= v_single;  -- variable to signal assignment
s_vector <= v_vector;  -- variable to signal assignment
v_single := s_single;  -- signal to variable assignment
v_vector := s_vector   -- signal to variable assignment
```

Loops

- A mechanism to perform repetitive assignments a fixed number of times or infinitely
- Useful in test benches for creating:
  - stimulus such as clocks or other periodic waveforms
  - a range of input vectors
- Loops are exclusively for sequential code ⇒ they can only be used inside a process
- There are several ways of describing loops

Infinite loops

- `LOOP` ⟶ the loop is repeated an infinite number of times

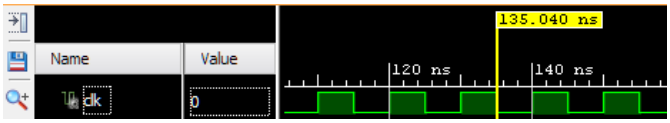
```
[label:] loop
      [sequential statements]
   exit [label] [WHEN condition]      -- optional exit statement
      [sequential statements]
   next [label] [WHEN condition]      -- optional next statement
      [sequential statements]
end loop[label];
```

- `exit` ⇒ forces the loop to end if condition = `true`
- `next` ⇒ skip the remaining signal assignments if condition = `true` ⟶ begins the next iteration of the loop

Infinite loops

- Example of an infinite `loop` to generate `clk` in a test bench

```
clk_proc: process
begin
   loop
      CLK <= not CLK;
      wait for 5 ns;       -- 10 ns period ⇒ 100MHz
   end loop;
end process;
```



Infinite loops with a stop condition

- `WHILE` / `LOOP` ⟶ loop is repeated until condition = `false`

```
[label:] WHILE condition LOOP
   [sequential statements]
END LOOP [label];
```

- e.g. ⟶ `loop` repeats while i < 10

```
WHILE (i < 10) LOOP
   WAIT UNTIL (clk'event AND clk='1');
   [sequential statements]
END LOOP;
```

Finite loops

- `FOR` / `LOOP` → loop is repeated a fixed number of times

```
[label:] FOR identifier IN range LOOP
   [sequential statements]
END LOOP [label];
```

- e.g. → loop is repeated unconditionally until `i=15` (16 times)

```
FOR i IN 0 TO 15 LOOP
   count <= count + 1;
   a <= count;
END LOOP;
```

  - It is also possible to execute a loop that counts down rather than up

```
FOR i IN 5 DOWNTO 0 LOOP
```


`exit` statement

- Example of a `loop` with an `exit` statement

```
FOR i IN 0 TO 255 LOOP
   CASE data(i) IS
      WHEN '0' => count := count + 1;
      WHEN OTHERS => EXIT;
   END CASE;
END LOOP;
```

In case data (i) <> '0', the loop is terminated even if i < 255

`next` statement

- Example of a `loop` with a `next` statement

```
FOR i IN 0 TO 255 LOOP
   [sequential statements 0]
   NEXT WHEN i = skip;        -- jumps to next iteration
   [sequential statements 1]
END LOOP;
```

In case data `i = skip`, `next` causes loop to skip one iteration ⇒ `[sequential statements 1]` are not executed this time

Create hardware with loops

- Loops are also synthesizable
  - while not common, loops may be used to create logic
  - the complete behavior of the module must be described by the loop
- Notice however
  - both limits of the range must be static
  - a declaration like `FOR i IN 0 TO choice LOOP`, where `choice` is an input (nonstatic) parameter, is not synthesizable



Practical hardware example with `loop`

- 8-bit unsigned adder with carry

```
ENTITY adder IS
   PORT ( a, b: IN STD_LOGIC_VECTOR (7 DOWNTO 0);
          cin: IN STD_LOGIC;
          s: OUT STD_LOGIC_VECTOR (7 DOWNTO 0);
          cout: OUT STD_LOGIC);
END adder;
```

Ex. 19

In this example, a structural approach is used: the structure of an 8-bit adder is described as a cascade of 1-bit adder modules repeated 8 times

Practical hardware example with `loop`

```
ARCHITECTURE adder OF adder IS
BEGIN
   PROCESS (a, b, cin)
      VARIABLE carry : STD_LOGIC_VECTOR (8 DOWNTO 0);
   BEGIN
      carry(0) := cin;
      FOR i IN 0 TO 7 LOOP
         s(i) <= a(i) XOR b(i) XOR carry(i);
         carry(i+1) := (a(i) AND b(i)) OR (a(i) AND
                       carry(i)) OR (b(i) AND carry(i));
      END LOOP;
      cout <= carry(8);
   END PROCESS;
END adder;
```

A needlessly complicated structural implementation!

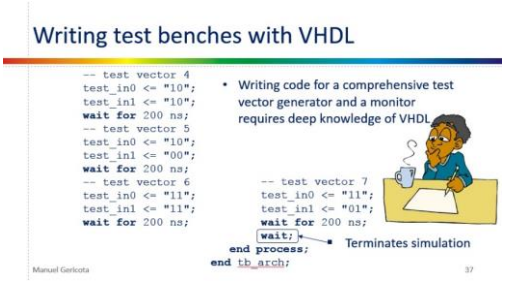


Logic simulation results



Creating test benches with loops – the intelligent way

- Remember how the test bench for the 2-bit comparator was created ?
  - all possible combinations generated and listed manually
- Loops enable to automate the process
  - all possible input combinations of a combinational circuit are generated automatically



A new test bench for the 2-bit comparator

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY  eq2_testbench IS
END  eq2_testbench;

ARCHITECTURE tb_arch of eq2_testbench IS
--Inputs
signal test_in0: std_logic_vector(1 downto 0) := (others => '0');
signal test_in1: std_logic_vector(1 downto 0) := (others => '0');
--Outputs
signal test_out : std_logic;
 --Internal
signal count : unsigned (3 downto 0) := (others => '0');
```

- all vector bits are initialized to zero
- all signals in the simulation should always be initialized

Ex. 20

A new test bench for the 2-bit comparator

```
BEGIN
   uut:entity work.eq2 (sop_arch)   -- Instantiate the uut
      PORT MAP (
         a => test_in0,
         b => test_in1,
         aeqb => test_out
         );
stim_proc: process                  -- Stimulus process
   begin
      for i in 0 to 15 loop
         test_in0 <= std_logic_vector (count (1 downto 0));
         test_in1 <= std_logic_vector (count (3 downto 2));
         count <= count + 1;
         wait for 100ns;
      end loop;
      wait;
   end process;
END;
```

All the 16 possible combinations of the two 2-bit inputs – the test vectors – are automatically generated using a single loop ⇒ the comparator is exhaustively tested

Static (default) values

- One good design practice is to replace “hard literals” with symbolic constants
  - it makes the code clear
  - it makes the code more adaptable and thus reusable in the future
  - it helps future maintenance and revision
- `constant` and `generic` declarations
  - serve to establish default values
  - are evaluated during preprocessing and thus require no physical circuit

`constant`

- Included in the architecture’s declaration section
- Assignment operator for a constant → ‘ := ’
- Syntax:

```
constant const_name: data_type := value_expression;
```

- e.g.

```
constant DATA_BIT: integer := 8;
constant DATA_RANGE: integer := 2*DATA_BIT - 1;
```

constant

- Hard literals may be substituted by constants to improve description readability
  - e.g. a 4-bit adder with carry-out bit

to convert the adder architecture in an 8-bit adder, 3 changes have to be made inside the code

```
architecture hard_arch of add_w_carry is
  signal a_ext, b_ext, sum_ext: unsigned (4 downto 0);
  begin
        a_ext <= unsigned ('0' & a);
        b_ext <= unsigned('0' & b);
        sum_ext <= a_ext + b_ext;
        sum <= std_logic_vector(sum_ext(3 downto 0));
        cout <= sum_ext (4) ;
end hard_arch;
```


constant

- with the use of a symbolic constant only one change is necessary

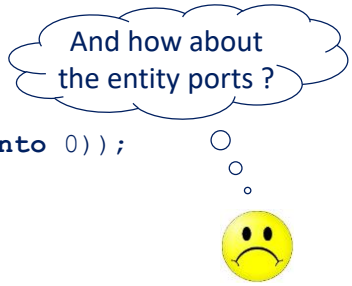
```
architecture const_arch of add_w_carry is
     constant N: integer := 4;
     signal a_ext, b_ext, sum_ext: unsigned (N downto 0);
begin
     a_ext <= unsigned('0' & a);
     b_ext <= unsigned ('0' & b);
     sum_ext <= a_ext + b_ext;
     sum <= std_logic_vector(sum_ext(N-1 downto 0));
     cout <= sum_ext(N);
end const_arch;
```

- the readability of the code is improved
- the code is easier to understand and maintain

constant

- with the use of a symbolic constant only one change is necessary

```
architecture const_arch of add_w_carry is
     constant N: integer := 4;
     signal a_ext, b_ext, sum_ext: unsigned (N downto 0);
begin
     a_ext <= unsigned('0' & a);
     b_ext <= unsigned ('0' & b);
     sum_ext <= a_ext + b_ext;
     sum <= std_logic_vector(sum_ext(N-1 downto 0));
     cout <= sum_ext(N);
end const_arch;
```



- the readability of the code is improved
- the code is easier to understand and maintain

generic

- To pass information into an `entity` and a `component`
  - declared inside an `entity` or `component` declaration, before the `port` declaration
  - may also be used inside the `architecture` body
  - cannot be modified inside the `architecture`
- Enables the creation of customizable modules ⇒ scalable code

`generic` sintax

```
entity entity_name is
generic (
      generic_name: data_type := default_values;
      ...
      generic_name: data_type := default_values
         );
port (
      port_name: mode data_type;
      ...
      );
end entity_name;
```

Assignment operator for a `generic`

A customizable adder with carry-out bit

```
library ieee;
use ieee. std_logic_1164. all;
use ieee. numeric_std. all;
entity gen_add_w_carry is
      generic (N: integer := 4);
      port (
         a, b: in std_logic_vector (N-1 downto 0);
         cout: out std_logic;
         sum: out std_logic_vector (N-1 downto 0)
      );
end gen_add_w_carry;
```

After being declared, `N` can be used in the `port` declaration and `architecture` body

Ex. 21

A customizable adder with carry-out bit

```
architecture arch of gen_add_w_carry is
signal a_ext, b_ext, sum_ext: unsigned (N downto 0);
begin
      a_ext <= unsigned('0' & a);
      b_ext <= unsigned('0' & b);
      sum_ext <= a_ext + b_ext;
      sum <= std_logic_vector(sum_ext(N-1 downto 0));
      cout <= std_logic (sum_ext (N));
end arch;
```

Changing the value of N, changes the width of the adder

Scalable code

- A generic provides a mechanism to create scalable code
  - the "width" of a circuit can be adjusted to meet a specific need
  - this makes code more portable ⇒ encourages design reuse
- If the adder is later used as a module in other design → the desired value of the generic may be assigned in `component` instantiation ⇒ generic mapping
- The default value is used if generic mapping is omitted

Component instantiation with `generic`

```
-- instantiate 8-bit adder
adder_8_unit: entity work.gen_add_w_carry(Behavioral)
generic map (N => 8)
port map (a=>a8, b=>b8, cout=>c8, sum=>sum8);

-- instantiate 16-bit adder
adder_16_unit: entity work.gen_add_w_carry(Behavioral)
generic map (N => 16)
port map (a=>a16, b=>b16, cout=>c16, sum=>sum16);

-- instantiate 4-bit adder
-- (generic mapping omitted, default value 4 used)
adder_4_unit: entity work.gen_add_w_carry(Behavioral)
port map (a=>a4, b=>b4, cout=>c4, sum=>sum4));
```

Ex. 22

In sum - assignment operators

- `<=` used to assign a value to a SIGNAL
- `:=` used to assign a value to a VARIABLE, CONSTANT, or GENERIC, and to set initial values
- `=>` used to assign values to individual vector elements, to map ports or with OTHERS

Packages

- IEEE standard package ⟶ implied when using VHDL
- User can create her/his own packages
  - types, signals, constants, variables, components declared in a package ⇒ visible in any architecture which uses the package ⇒ no need to be declared again
  - e.g. signals in package declarations ⇒ global signals
- User packages ⟶ stored in the default library `work`
  - add the new library declaration if using the items therein declared
  - e.g. `use work.<package_name>.all;`

Package declarations

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.ALL; -- if using signed or unsigned values
package <package_name> is
```

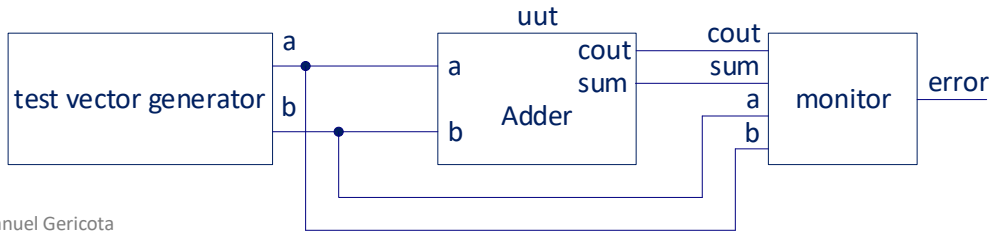
- type declarations
- signal declarations
- constant declarations
- variable declarations
- component declarations
- functions / procedures

```
component component_name
    generic (generic_list);
    port (port_list);
end component;
```

```
end <Package_Name>;
```

Logic simulation results with automatic verification

- More complicated circuits ⇒ automatic detection of errors
- Sketch of the customizable adder with carry-out bit
  - uut ⟶ unit under test
  - test vector generator block ⟶ generates testing input patterns
  - monitor block ⟶ examines the output responses



Defect, fault, error, and failure concepts

- Defect ⟶ something physical – e.g. an open or a short in a wire
- Fault ⟶ the logical manifestation of a defect - a short between a wire and ground manifests itself as a stuck-at logic '0' (the logic value in the wire is always '0' – s@0)
- Error ⟶ a deviation from correctness or accuracy in computation, which occurs as a result of a fault – e.g. if the s@0 wire is part of an adder module ⇒ one of the bits will always be zero, independently of the operands ⇒ actual result ≠ expected result
- A system is said to have a failure if it fails to perform some action which is due or expected in a specified period of time

Test vector generation

- Deterministic - specific to uut developed via ATPG (Automatic Test Pattern Generators) or fault simulation
- Algorithmic - used to test regular structures and specific fault models, e.g. adders
- Exhaustive - all possible input test patterns, e.g. combinational logic
- Pseudo-exhaustive - each partitioned sub-unit is exhaustively tested
- Pseudo-random - properties similar to random sequences but repeatable

Test benches

- The code consists of
  - a component instantiation statement → creates an instance of the adder
  - a stimulus process statement → generates the sequence of test patterns applied to the adder's input ports
  - a result process statement → computes the correct sum result
  - a verification process statement → compares the adder's output with the sum result

Stimulus process

```
stim_proc: process
   begin
   for i in 0 to 2**N-1 loop          -- apply all 2^(N-1)
      a <= count(N-1 downto 0);       -- possible combinations
      b <= count(2*N-1 downto N);     -- to adder's inputs
      count <= count + 1;
      wait for 100ns;                 -- each 100 ns
   end loop;
end process;
```

N → `generic` that defines the adder's width

A counter creates all possible combinations of the inputs ⇒ exhaustive test of the adder, no matter its width → the test vector generation block is also customizable

Result process

- The monitor computes the correct value for each combination of the inputs…

```
result_proc: process (a, b, cout, sum)
   -- each time the adder's sum output port changes value
   -- the new sum of the inputs is computed and
   -- the adder's sum and carry outputs are concatenated
   begin
      sum_verif <= ('0' & a) + ('0' & b) ;
      sum_result <= cout & sum;
end process;
```

Monitor - verification process

- … and compares it with the uut outputs, checking for errors

```
verif_proc: process (sum_result, sum_verif)
-- each time a new sum value is computed, it is
-- compared with the adder's output results
-- and an error signal is generated if they are different
begin
   if sum_result /= sum_verif
      then error <= '1';
      else error <='0';
   end if;
end process;
```


Errors in the code and fault injections

- A change in the VHDL adder description may lead to wrong results, depending on the input values

Ex. 23

```
cout <= sum_ext(N); -- correct operation (original code)
cout <= sum_ext(N-1); -- error introduction
```

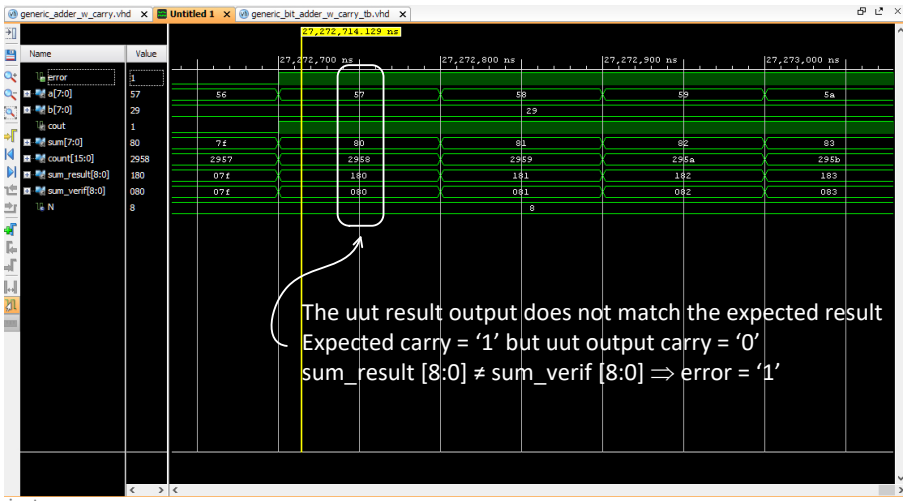
- Faults, simulating physical defects, may also be injected

Ex. 24

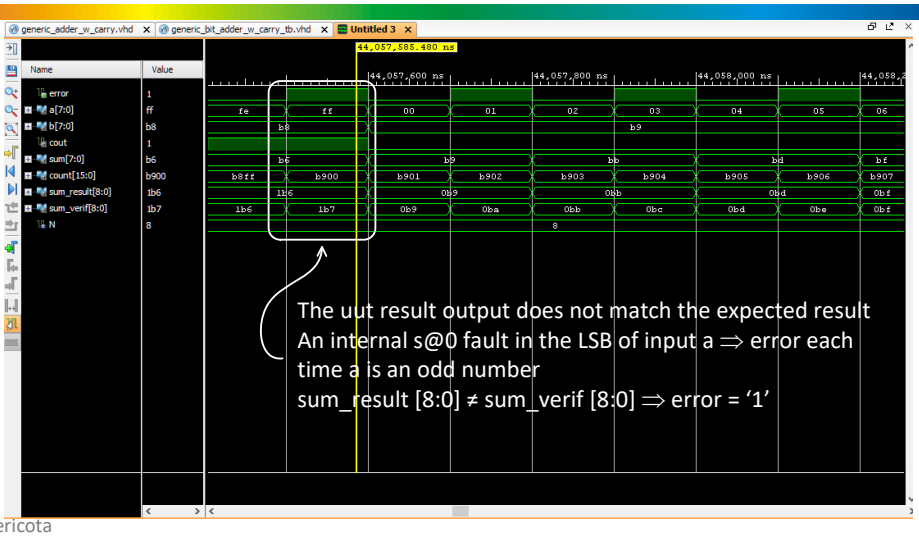
```
a_ext <= unsigned('0' & a); -- correct operation
a_ext <= unsigned('0' & a(N-1 downto 1) & '0'); -- fault
                  -- injection the LSB of a is stuck-at 0
```

- Depending on the values of the adder's inputs, the monitor detects and signals the error

Logic simulation of the error condition



Logic simulation of the fault condition



Sequential circuit

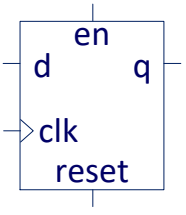
- A circuit with a memory stores its internal state

  output of a sequential circuit = f (*inputs*, *internal state*)

- Synchronous design methodology ⟶ the most commonly used practice in designing sequential circuits
  - all storage elements are controlled (i. e., synchronized) by a global clock signal
  - data is sampled and stored at the rising or falling edge (or both!) of the clock signal
  - storage components are separated from the rest of the circuit ⟶ simplify the design process

D flip-flops and registers

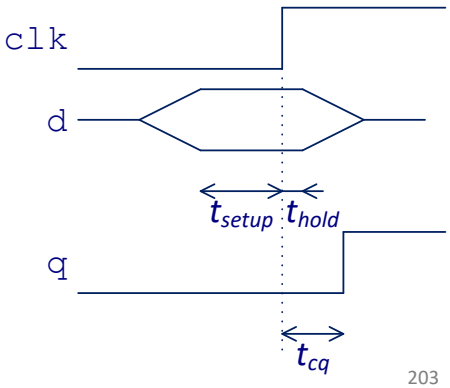
- D-type flip-flop (D FF) ⟶ stores circuit's internal state
  - input signal *d* ⟶sampled at the rising (falling) edge of the *clk* signal and stored in the FF
  - optionally D FF may have
    - a synchronous or asynchronous reset signal ⟶ clear the FF to '0' or '1'
    - a synchronous enable signal ⟶ disables the storage of new values
  - a D FF provides 1-bit storage
    - a collection of D FFs ⟶ register ⟹ to store multiple bits



clk	reset	en	q*
-	1	-	0
0	0	-	q
1	0	-	q
↑	0	0	q
↑	0	1	d

D FF timing parameters

- Three main timing parameters
  - T_{cq} (*clock-to-q*) delay → time required to propagate the value of *d* to *q* at the rising edge of the clock signal
  - the *d* signal must be stable around the sampling edge to prevent the FF from entering a metastable state
  - T_{setup} (*setup time*) and T_{hold} (*hold time*) specify the time intervals before and after the sampling edge



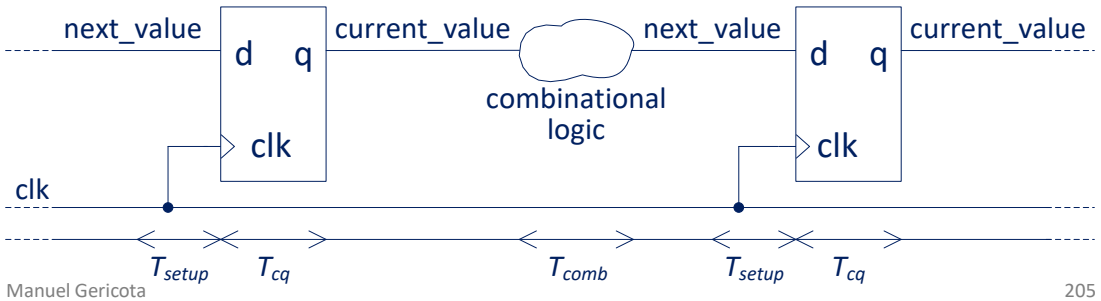
Metastability

- Metastable state
  - the output hovers for an unbounded time in an unstable equilibrium at a value between the high and low states
  - the output transition to a defined ‘0’ or ‘1’ is delayed beyond the specified t_{cq}
  - a stable logic level is not achieved within the time required for the correct operation of the circuit

D FF timing parameters

- Sequential circuit ⇒ essential to ensure system timing does not violate the setup and hold time constraints

$$T_{clock} = T_{cq} + T_{comb} + T_{setup}$$



Maximal operating frequency

- Timing of a sequential circuit is characterized by f_{max} ⇒ the *maximal clock frequency* → specifies how fast the circuit can operate
  - the reciprocal of f_{max} specifies T_{clock} (=1/ f_{max}) → the minimal clock period → the interval between two sampling edges of the clock
  - correct operation ⇒ the next value must be generated and stabilized within this interval

$$f_{max} = \frac{1}{T_{clock}} = \frac{1}{T_{cq} + T_{comb} + T_{setup}}$$

Maximal operating frequency

- During synthesis ⟶ the tool analyses the synthesized circuit and determines f_{max}
  - the desired frequency should be specified as a synthesis constraint
  - the software tries to reach an $f_{max} \geq$ user desired frequency
- For a 100 MHz clock (T = 10 ns), 50% duty-cycle, the following line may be added to the constraint file

maximal period — duty-cycle

```
create_clock -period 10.000 -name clk -waveform {0.000 5.000}
[get_ports clk]
```


Pipelining

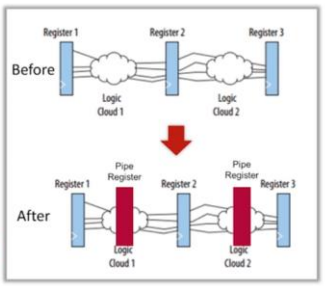
- Introduced in Vivado 2017.3 (Oct 2017)

New Automatic Pipelining

- Helps analyze potential for improvement if more registers were available
- Option set as a block_synth value
  - AUTO_PIPELINING
- Adds registers to improve timing
  - Detects critical paths with greatest number of logic levels
  - User can add pipeline levels to be pushed into logic
  - The tool balances rest of the logic for best performance

Example for setting up BLOCK_SYNTH in .xdc

```
set_property BLOCK_SYNTH.AUTO_PIPELINING 1 [get_cells a]
```



Page 27 © Copyright 2017 Xilinx XILINX ALL PROGRAMMABLE.

D FF with synchronous enable

```
entity d_ff_en is
   port (
      clk, reset, en, d: in std_logic;
      q: out std_logic
   );
end d_ff_en;
architecture arch of d_ff_en is
begin
   process(clk, reset)
      begin
         if (reset='1') then
            q <= '0';
         elsif(clk'event and clk='1') then
            if (en='1') then
               q <= d;
            end if;
         end if;
   end process;
end arch;
```

- the `en` signal enables the D FF to sample the input value at the rising edge of the clock ⇒ the `en` signal is synchronous
- if the `en` signal is not asserted, the D FF keeps its previous value

Ex. 25

Uses of the D FF with synchronous enable

- To maintain synchronism between a fast subsystem and a slow subsystem
  - e.g. assuming that the operation rates of a fast and a slow subsystem are 50 MHz and 1 MHz, instead of using a derived 1-MHz clock to drive the slow subsystem, a periodic enable tick that is asserted one clock cycle every 50 clock cycles can be generated
  - the slow subsystem is disabled (i.e., keep the previous state) for the remaining 49 clock cycles
- To eliminate a gated clock signal
- To create slow counters

More about sequential circuits

- Based on the characteristics of the next-state logic, sequential circuits may be divided into three categories:
  - Regular sequential circuit → state transitions exhibit a "regular" pattern, e. g. counters or shift registers
    - next-state logic constructed primarily by a predesigned, "regular" component, e.g. an incrementor or shifter
  - FSM (finite state machines) → state transitions do not exhibit a simple, repetitive pattern
    - next-state logic constructed by "random logic" and synthesized from scratch → a random sequential circuit

Sequential circuit

- FSMD (FSM with data path) → the circuit consists of a regular sequential circuit + an FSM
  - two parts → data path + control path = an FSMD
  - implements algorithms represented by register-transfer (RT) methodology (RTL – Register Transfer Level) → describes system operation by a sequence of data transfers and manipulations among registers

$$r_{dest} \leftarrow f(r_{src1}, r_{src2}, \ldots, r_{srcn})$$

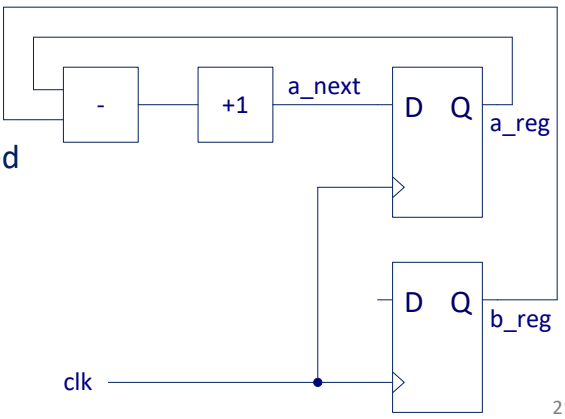
r_{dest}	destination register
r_{src}	source register
$f(.)$	operation to be performed → combinational circuit

FSMD

- the destination register is updated at the next rising edge of the clock
- e.g.
  - r1 ⟵ 1 — a constant 1 is stored in register r1
  - r1 ⟵ r1 — the content of register r1 is written back to itself
  - r2 ⟵ r2 >> 3 — the content of register r2 is shifted right three positions and then written back to itself
  - r2 ⟵ r1 — the content of register r1 is transferred to register r2
  - i ⟵ i + 1 — the content of register i is incremented by 1 and the result is written back to itself
  - d ⟵ s1 + s2 + s3 — the sum of s1, s2, and s3 registers'content is written to register d
  - y ⟵ a*a — the result of a^2 content is written to register y

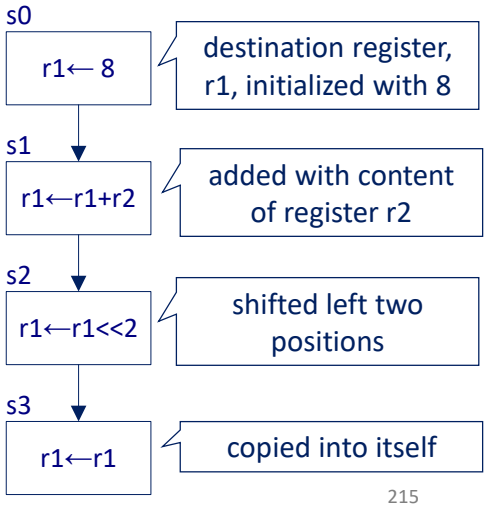
FSMD

- a single RT operation can be implemented by constructing a combinational circuit for the *f* (.) function + connecting the input and output of the registers
- e.g. a ⟵ a-b+1
- the RT operation is synchronized by an embedded clock
- the result from the *f* (.) function is not stored to the destination register until the next rising edge of the clock



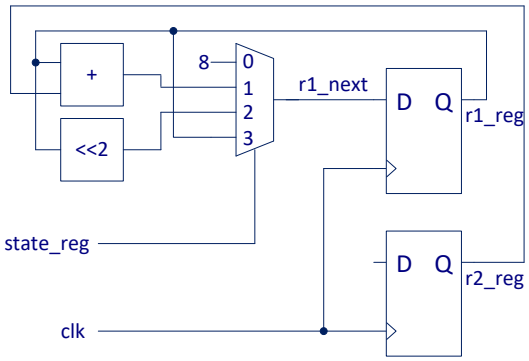
ASMD chart

- An RT methodology –based circuit specifies which RT operations should be executed in each step
- RT operations performed synchronously with the FSM state transition
  - RT operations are incorporated in an ASMD, an ASM with Data path chart
  - register r1 must be specified in all states



FSMD implementation

- yet, if r1 is kept unchanged ⇒ r1 ← r1 default RT operation → no need to include it in the ASMD chart
- implementation involves a multiplexing circuit to route the desired next value to the destination register
- e.g. the previous segment can be implemented by a 4-to-1 multiplexer
  - the current state (i.e., the output of the state register) of the FSM controls the selection signal of the multiplexer ⇒ chooses the result of the desired RT operation



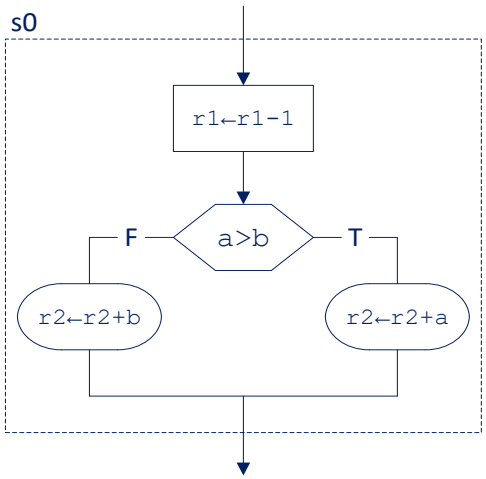
RT operations on a conditional output box

- Operations can be specified in conditional output box, as register `r2`
- Depending on condition

`a > b`

⇓

`r2 ← r2+a` or `r2 ← r2+b`

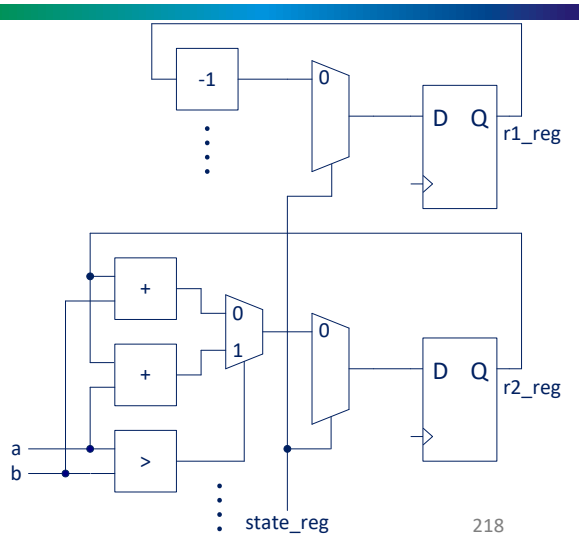


RT operations on a conditional output box

- Operations

`a>b` `r2+a` `r2+b`

are carried out in parallel
- The multiplexer routes the desired value to `r2`



Decision box with a register

- ASMD chart is similar to a normal flowchart
- The main difference → the RT operation in an ASMD chart is controlled by an embedded clock signal
  - the destination register is updated when the FSMD exits the current ASMD block, but not within the block
- the `r ← r-1` operation actually means:

```
r_next <= r_reg - 1;
r_reg <= r_next
```

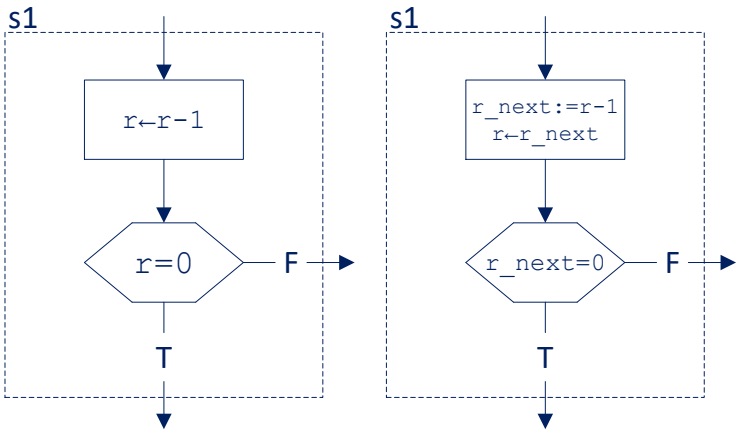
at the rising edge of the clock (i.e., when the FSMD exits the current block)

Decision box with a register

- This "delayed store" may introduce subtle errors when a register is used in a decision box
- If a register `r` is decremented in the state box and used in the decision box ⇒ the old content of `r` is used for comparison in the decision box instead of the new one because the `r` register is not updated until the FSMD exits the block
- If the new value of `r` is desired for immediate comparison ⇒ the output of the combinational logic (i.e., `r_next`) should be used in the decision box

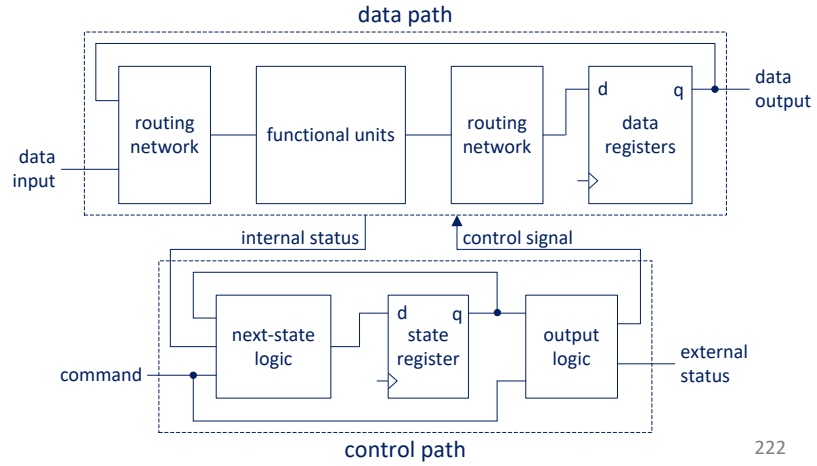
Decision box with a register

- Note the use of the := notation, as in `r_next:=r-1`
  → indicates the immediate assignment of `r_next`



Block diagram of an FSMD

- Data path + control path
  - both paths are sequential circuits controlled by the same clock ⇒ the FSMD is a synchronous system



Block diagram of an FSMD

- Data path
  - performs the required RT operations
    - data registers: store the intermediate computation results
    - functional units: perform the functions specified by the RT operations
    - routing network: routes data between storage registers and functional units
  - follows the control signal to perform the desired RT operations and generates the internal status signal

Block diagram of an FSMD

- Control path
  - an FSM
    - contains a state register, next-state logic, and output logic
    - uses the external command signal and the data path's status signal as the input
    - generates the control signal to control the data path operation
    - reads the external status signal that indicate the status of the FSMD operation

Code development of an FSMD

- Explicit description ⟶ the control FSM is separated from the key data path components
  - from an ASMD chart
    1. identify the key components in the data path + the associated control signals
    2. describe those components in individual code segments
- Implicit description ⟶ the RT operations are embedded within the FSM control
  - RT operations are listed with the corresponding FSM state

Explicit *vs* Implicit description

```
case
      when s1
         in1 <= a;
         in2 <= b;
         d1_next <= m_out;
      when s2
         in1 <= b;
         in2 <= c;
         d2_next <= m_out;
end case;
m_out <= in1 * in2;
```

The datapath multiplier is explicitely separated from the control path FSM

```
case
   when s1
      d1_next <= a * b;
   when s2
      d2_next <= b * c;
end case;
```

The multiplier in the datapath is embedded in the control path's FSM description

An example of a FSMD – the Double-Dabble Binary-to-BCD Conversion Algorithm*

- For a n-bit number stored in a n-bit register
  - create a register with a length equal to n-bit plus the number of bits needed to store its representation in BCD
  - store the original value in the least significant bits of the new register and initialize the new bits to zero
  - divide the extra space in BCD digits (4-bits each digit) starting from the MSB → e.g. 0000 0000 0000 10110010
  - left-shift the entire register space one bit if all BCD digits ≤ 4

* Rhyne, V.T., "Serial Binary-to-Decimal and Decimal-to-Binary Conversion," *IEEE Transactions on Computers*, Vol. C-19, No. 9, pp. 808-812, Sept. 1970

Binary-to-BCD conversion algorithm

- if any BCD digit > 4 ⇒ increment it by 3 before the left-shift is done
  - the increment ensures that a value of 5, incremented and left-shifted, becomes 16, correctly "carrying" into the next BCD digit
- repeat the shifting many times as the number of bits of the original number
- e.g.

  `0000 0000 1011` → begin – 4 bits – $1011_2 \Rightarrow 11_{10}$

  `0000 0101 1xxx` → after 3 shifts the 1st BCD is greater than 4

  `0000 1000 1xxx` → add 3 to the 1st BCD digit

  `0001 0001 xxxx` → after the 4th and last shift – 11BCD ⇒ 11_{10}

Binary-to-BCD conversion algorithm

1. 8-bit initial value $10110110_2 \rightarrow 182_{10}$
2. add enough BCD digits to the left and apply the algorithm
3. `0000 0000 0000 10110110` → begin – 8 bits (182_{10}) ⇒ 3 BCD digits
4. `0000 0000 0001 0110110x` → 1st shift
5. `0000 0000 0010 110110xx` → 2nd shift
6. `0000 0000 0101 10110xxx` → after the 3rd shift the 1st BCD > 4
7. `0000 0000 1000 10110xxx` → result of adding 3 to 1st BCD digit
8. `0000 0001 0001 0110xxxx` → 4th shift
9. `0000 0010 0010 110xxxxx` → 5th shift
10. `0000 0100 0101 10xxxxxx` → after the 6th shift the 1st BCD > 4
11. `0000 0100 1000 10xxxxxx` → result of adding 3 to 1st BCD digit
12. `0000 1001 0001 0xxxxxxx` → after the 7th shift the 1st BCD > 4
13. `0000 1100 0001 0xxxxxxx` → result of adding 3 to 2nd BCD digit
14. `0001 1000 0010 xxxxxxxx` → 8th and last shift (**n** = 8 bits)

Another example

Binary-to-BCD conversion circuit ⇒ an FSMD

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity bin_to_bcd is
    port(clk : in  STD_LOGIC;
         reset : in  STD_LOGIC;
         start : in  STD_LOGIC;
         bin : in  STD_LOGIC_VECTOR (12 downto 0);
         done_tick : out  STD_LOGIC;
         ready : out  STD_LOGIC;
         bcd0 : out  STD_LOGIC_VECTOR (3 downto 0);
         bcd1 : out  STD_LOGIC_VECTOR (3 downto 0);
         bcd2 : out  STD_LOGIC_VECTOR (3 downto 0);
         bcd3 : out  STD_LOGIC_VECTOR (3 downto 0));
end bin_to_bcd;
```

reset signal for register initialization
start signal to start conversion
done_tick signals conversion end
ready signals available data

13-bit binary numbers ⇒ decimal number ≤ 8191 ⇒ 4 BCD digits

Binary-to-BCD conversion circuit - signals & types

```
architecture Behavioral of bin_to_bcd is

   type state_type is (idle, op, done);
      -- user-defined enumerated data type for state assignment
   signal state_reg, state_next: state_type;

   signal p2s_reg, p2s_next: std_logic_vector (12 downto 0);
   signal n_reg, n_next: unsigned (3 downto 0);
   signal bcd3_reg, bcd2_reg, bcd1_reg, bcd0_reg: unsigned (3
         downto 0);     -- BCD digit registers

   signal bcd3_next, bcd2_next, bcd1_next, bcd0_next: unsigned(3
         downto 0);     -- BCD next digit registers value

   signal bcd3_tmp, bcd2_tmp, bcd1_tmp, bcd0_tmp: unsigned(3
         downto 0);     -- BCD temporary calculation values
```


Binary-to-BCD conversion circuit – state & data registers

```
begin          -- state and data registers update
process (clk, reset)
begin
   if reset='1' then
      state_reg <= idle;
      p2s_reg <= (others=> '0');
      n_reg <= (others=> '0');
      bcd3_reg <= (others=> '0');
      bcd2_reg <= (others=> '0');
      bcd1_reg <= (others=> '0');
      bcd0_reg <= (others=> '0');
   elsif rising_edge(clk) then
      state_reg <= state_next;
      p2s_reg <= p2s_next;
      n_reg <= n_next;
      bcd3_reg <= bcd3_next;
      bcd2_reg <= bcd2_next;
      bcd1_reg <= bcd1_next;
      bcd0_reg <= bcd0_next;
   end if;
end process;
```

Binary-to-BCD conversion circuit – next-state logic & data path operations

```
-- fsmd next-state logic / data path operations
process (state_reg,start,p2s_reg,n_reg,n_next,bin,
         bcd0_reg,bcd1_reg,bcd2_reg,bcd3_reg,
         bcd0_tmp,bcd1_tmp,bcd2_tmp,bcd3_tmp)
begin
   state_next <= state_reg;    -- initialize output signals
   ready <= '0';               -- invalid output BCD digits
   done_tick <= '0';           -- conversion in progress

   p2s_next <= p2s_reg;
   bcd0_next <= bcd0_reg;
   bcd1_next <= bcd1_reg;
   bcd2_next <= bcd2_reg;
   bcd3_next <= bcd3_reg;
   n_next <= n_reg;
```


Binary-to-BCD conversion circuit – next-state logic & data path operations

```
case state_reg is
   when idle =>
      ready <= '1';         -- valid output BCD digits available
      if start='1' then     -- start conversion
         bcd3_next <= (others=> '0');     -- clean all BCD
         bcd2_next <= (others=> '0');     -- registers to start
         bcd1_next <= (others=> '0');     -- new conversion
         bcd0_next <= (others=> '0');
         n_next <="1101";     -- number of shifts = binary length
         p2s_next <= bin;     -- upload input binary number
         state_next <= op;    -- next-state -> operation
      end if;
```

Binary-to-BCD conversion circuit – next-state logic & data path operations

```
   when op =>
      p2s_next <= p2s_reg (11 downto 0) & '0'; -- shift binary left
         -- shift 4 BCD digits left 1-bit
      bcd0_next <= bcd0_tmp(2 downto 0) & p2s_reg(12);
      bcd1_next <= bcd1_tmp(2 downto 0) & bcd0_tmp(3);
      bcd2_next <= bcd2_tmp(2 downto 0) & bcd1_tmp(3);
      bcd3_next <= bcd3_tmp(2 downto 0) & bcd2_tmp(3);
      n_next <= n_reg - 1;    -- decrements number of shifts
      if (n_next=0) then      -- no more shifts of the
         state_next <= done;  -- binary input number
      end if;
   when done =>
      state_next <= idle;
      done_tick <= '1';       -- conversion done
   end case;
end process;
```


Binary-to-BCD conversion circuit – data path functional units

```
   -- data path function units -> four BCD adjustment circuits
   bcd0_tmp <= bcd0_reg + 3 when bcd0_reg > 4 else
               bcd0_reg;
   bcd1_tmp <= bcd1_reg + 3 when bcd1_reg > 4 else
               bcd1_reg;
   bcd2_tmp <= bcd2_reg + 3 when bcd2_reg > 4 else
               bcd2_reg;
   bcd3_tmp <= bcd3_reg + 3 when bcd3_reg > 4 else
               bcd3_reg;
   bcd0 <= std_logic_vector(bcd0_reg);        -- Outputs
   bcd1 <= std_logic_vector(bcd1_reg);
   bcd2 <= std_logic_vector(bcd2_reg);
   bcd3 <= std_logic_vector(bcd3_reg);
end Behavioral;
```

Ex. 26

Registers

- A collection of D FFs controlled by the same clock
  - registers have a synchronous or asynchronous reset signal + optionally a synchronous enable signal
  - the code is identical to that of a D FF except that the array data type, `std_logic_vector`, is needed for the relevant input and output signals

Registers

```
entity reg_reset is
   port(
      clk, reset: in std_logic;
      d: in std_logic_vector (7 downto 0);
      q: out std_logic_vector (7 downto 0)
   );
end reg_reset;
architecture arch of reg_reset is
begin
   process(clk, reset)
   begin
      if (reset='1') then
         q <= (others=>'0');
      elsif(clk'event and clk='1') then
         q <= d;
      end if;
   end process;
end arch;
```

Remember that the expression `(others=>'0')` means that all elements are assigned to '0' and is equivalent to "0000 0000" in this case

Ex. 27

Composite types

- Composite types have multiple elements
- Two classes of composite types
  - array types
  - record types
- The difference between arrays and records $\Rightarrow$ all elements of an array must be of the same type $\leftrightarrow$ each element of a record can be of a different type

Composite types - `array`

- e.g.
  - an array of 32 lines, each line composed by an 8-bit vector

```
type reg_file_type is array (31 downto 0) of
    std_logic_vector (7 downto 0);
```

  - an array of 64 lines, each line composed by integers

```
type address_word is array (0 to 63) of integer;
```

Register files

- A collection of registers with one input port and one or more output ports
  - a write address signal, `w_addr` → specifies where to store data
  - a read address signal, `r_addr` → specifies where to retrieve data
  - generally used as fast, temporary storage
- The example shows a customizable 2^W-by-B register file
  - two generics in this design
    - the `W` generic specifies the number of address bits ⇒ 2^W words in the file
    - the `B` generic specifies the number of bits in a word

Customizable register file

Ex. 28

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;
entity reg_file is
   generic (
      B: integer := 8;     -- number of bits
      W: integer := 2      -- number of address bits
      );
   port (
      clk, reset: in std_logic;
      wr_en: in std_logic;
      w_addr, r_addr: in std_logic_vector (W-1 downto 0);
      w_data: in std_logic_vector (B-1 downto 0);
      r_data: out std_logic_vector (B-1 downto 0)
   );
end reg_file;
```

Customizable register file

```
architecture arch of reg_file is
   type reg_file_type is array (2**W-1 downto 0) of
           std_logic_vector (B-1 downto 0);
   signal array_reg: reg_file_type;
   begin
      process(clk,reset)
      begin
         if (reset='1') then
            array_reg <= (others=>(others=> '0'));
         elsif (clk'event and clk='1') then
            if wr_en='1' then
               array_reg(to_integer(unsigned(w_addr))) <= w_data;
            end if;
         end if;
      end process;
   -- read port
   r_data <= array_reg(to_integer(unsigned(r_addr)));
end arch;
```

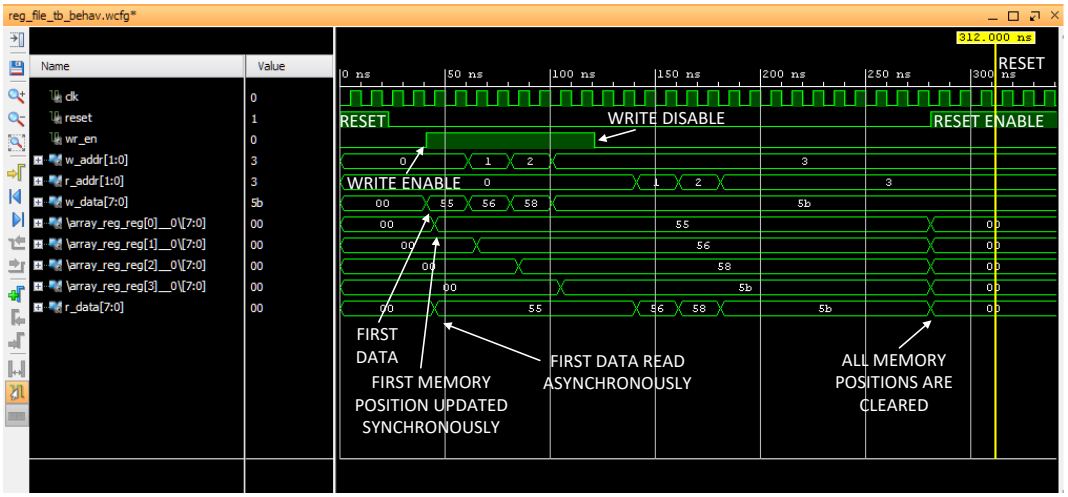
Composite `array` type

all bits in all words = '0'

Customizable register file

- The code includes several new features
  - a user-defined composite type array, `reg_file_type`, is used to define the register file size
    1. defined by a composite type statement
    2. used by the `array_reg` signal
  - a signal is used as an index to access an element in the array, as in `array_reg ( .. w_addr .. )`
  - the `array_reg (...) <=` … and … `<= array_reg (...)` statements infer decoding and multiplexing logic, respectively

Logic simulation of the register file



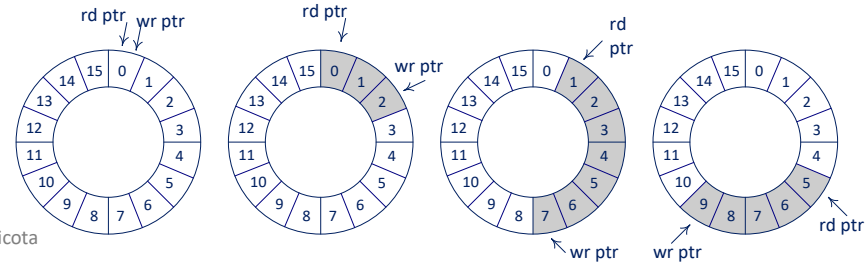
Multiple port register files

- Multiple read ports may be added to retrieve multiple data words at the same time
- e.g.

```
r_data2 <= array_reg(to_integer(unsigned(r_addr_2)));
r_data3 <= array_reg(to_integer(unsigned(r_addr_3)));
...
r_datan <= array_reg(to_integer(unsigned(r_addr_n)));
```

FIFO (first-in first-out) buffer

- FIFO buffer = control circuit + a register file
  - register file arranged as a circular queue with two pointers
    - the *write pointer* ⟶ points to the head of the queue
    - the *read pointer* ⟶ points to the tail of the queue
  - the corresponding pointer advances one position for each write or read operation



FIFO buffer

Ex. 29

- A FIFO buffer usually contains two status signals
  - full ⟶ indicates that the FIFO cannot be written
  - empty ⟶ indicates that the FIFO cannot be read

  } one of the two conditions occurs when the read pointer is equal to the write pointer
- A mechanism is needed to distinguish the two conditions
  - keep track of the last memory operation ⟶ read or write
  - check if the address of both read pointer and write pointer are equal
  - if the last operation was a read operation, the buffer is empty
  - if the last operation was a write operation, the buffer is full

Composite types - `record`

- Used to simplify entities and port maps in VHDL
- May contain elements of different types (std_logic, integer, unsigned, …)
- Kept in package files → visible across multiple files
- Signals defined as records can be initialized
- Mostly used to define a new VHDL type ⊂ any group of desired signals
  - useful for large interfaces always with the same list of signals
  - simply define the record type in a single package file
  - use the package file for any entity that makes use of that record type

Composite types - `record`

- `record` sintax

```
type record_type_name is record
    element_name : element type;
    element_name : element type;
    . . .
end record record_type_name;
```

- `record` initialization example

```
constant ini_record_name : record_type_name := (
    ( element_name => '0',
    ( element_name => (others => '0'),
    . . . );
```

e.g. FIFO buffer entity

Ex. 29

```
entity fifo is
generic ( B: integer := 8;    -- number of bits per memory cell
          W: integer := 2     -- number of address bits (number of
        );                    -- memory cells = 2^W)
port  (  clk : in std_logic;
         reset : in std_logic;      -- reset (active high)
         wr_data : in std_logic_vector ((B-1) downto 0);-- data in
         wr_en : in std_logic;      -- write enable (active high)
         rd_en : in std_logic;      -- read enable (active high)
         full_buf : out std_logic;  -- fifo full (active high)
         empty_buf : out std_logic; -- fifo empty (active high)
         rd_data : out std_logic_vector ((B-1) downto 0)-- data out
      );
end fifo;
```


Composite types - `record`

Ex. 30

- Example applied to the FIFO buffer
  - fixed data bus

Note: generic packages, part of VHDL 2008, are not yet supported by Vivado

```
type fifo_ports_in  is record                       -- input ports
         wr_data : std_logic_vector (7 DOWNTO 0); -- data in
         wr_en : std_logic;                       -- write enable
         rd_en : std_logic;                       -- read enable
end record fifo_ports_in;
type fifo_ports_out  is record                      -- output ports
         full_buf : std_logic;                    -- fifo full
         empty_buf : std_logic;                   -- fifo empty
         rd_data : std_logic_vector (7 DOWNTO 0); -- data out
end record fifo_ports_out;
```

Composite types - record

- Initialization of the record signals

```
constant fifo_ports_in_init : fifo_ports_in :=
         (wr_data => (others => '0'),    -- write bus
          wr_en => '1',                  -- write enable
          rd_en => '0'                   -- read
          );

constant fifo_ports_out_init : fifo_ports_out :=
         (full_buf => '0',               -- fifo full flag
          empty_buf => '1',              -- fifo empty flag
          rd_data => (others => '0')     -- read bus
         );
```


Composite types - record

- When declared in a package, add it to the design to be able to use the port signals herein declared

```
use work.my_records.all;         -- package with the record types
entity fifo is
port(
     clk : in std_logic;
     reset : in std_logic;
    -- input ports and their initialization
     in_fifo : in  fifo_ports_in  := fifo_ports_in_init;
    -- output ports and their initialization
     out_fifo : out fifo_ports_out := fifo_ports_out_init
    );
end fifo;
```

Composite types - `record`

- Invoke individual record signals adding the new entity name separated by ‘ . ’ from the record element name
  - Sintax

    `<new_entity_name>.<element_name>`
  - e.g.

    `in_fifo.wr_en`

    `out_fifo.full_buf <= full;`
- Records may also be used in the testbench, adding the package to the testench file

Functions

- A subprogram ⟶ defines a sequential algorithm
  - take one or more inputs, and always return a single output value
    - input arguments are read only inside the function
  - sequential code (like processes) ⟶ sequential statements `if`, `case` and `loop`; `wait` is not allowed
  - `SIGNAL` and `COMPONENT` are not synthesizable
  - intended mainly for LIBRARY allocation ⇒ usually placed in a `package`
    - store code to be reused or shared among projects
    - if desired, may be part of the main code
  - aimed to deal with commonly used operations

Function syntax

```
FUNCTION function_name [<parameter list>] RETURN data_type IS
   [declarations]
BEGIN
   (sequential statements)
END function_name;
```

- Parameter list → specifies function's input parameters (only IN)
  - any number of parameters, even zero
  - any synthesizable data type, but with no range – ~~RANGE~~ ← INTEGER, or ~~TO/DOWNTO~~ ← STD_LOGIC_VECTOR, UNSIGNED, SIGNED
  - only CONSTANT (by default) and SIGNAL; VARIABLE not allowed
    - <parameter list> = [CONSTANT] constant_name: constant_type
    - <parameter list> = SIGNAL signal_name: signal_type
  - same rules apply to RETURN data type

Function call

- A function call is an expression →
  `function-name (list-of-actual-values)`
- e.g.
  - `x <= add (a,b)` → calls function `add` to add `a` to `b` and assigns the result to `x` (`x` is a signal)
  - `y := maximum (a,b)` → calls function `maximum` to return the largest of `a` and `b` and assigns the result to `y` (`y` is a variable)
- Function calls may be associated to concurrent of sequential statements
  - `if y > maximum (a,b)` → associated to a sequential statement

Example of an 8-bit adder function for `std_logic_vector` signals

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

package my_functions is

   -- function declaration

   function add_function (signal a,b : std_logic_vector) return
         std_logic_vector;

end my_functions;
```

Package declaration

Ex. 31

Example of an 8-bit adder function for `std_logic_vector` signals

```
package body my_functions is

   function add_function (signal a,b : std_logic_vector)
        return std_logic_vector is

   variable result: std_logic_vector (8 downto 0);
   begin
      result := std_logic_vector (unsigned ('0' & a) +
                  unsigned('0' & b));
      return result;
   end add_function;

end my_functions;
```

Package body

input parameters

function statements

Procedures

- A subprogram → similar to a `function`
  - main difference → may return none or more than one value using parameters of mode `OUT` and `INOUT`
  - take one or more inputs, outputs or bidirectional arguments
  - sequential code (like processes) → sequential statements `if`, `case` and `loop`; it is allowed the use of `wait` statements
  - `COMPONENT` is not synthesizable
  - Signals may be declared if the `PROCEDURE` is declared inside a `PROCESS`
  - edge detection operations not allowed inside a `PROCEDURE`
    - a synthesizable `PROCEDURE` should not infer registers

`Procedure` syntax

```
PROCEDURE procedure_name [<parameter list>] IS
   [declarations]
BEGIN
   (sequential statements)
END procedure_name;
```

- Parameter list → specifies function's IN, OUT, INOUT parameters
  - any number of parameters
  - any synthesizable data type
  - `CONSTANT` (by default for IN parameters), `SIGNAL` or `VARIABLE` (by default for OUT and INOUT parameters)
    - `<parameter list> = CONSTANT constant_name: constant_type`
    - `<parameter list> = SIGNAL signal_name: signal_type`
    - `<parameter list> = VARIABLE variable_name: variable_type`

`Procedure` call

- A `PROCEDURE` call is a statement on its own →
  `procedure-name (list-of-actual-values)`
  - it may appear by itself or associated to a concurrent or sequential statement
- e.g.
  - `add (a,b,sum)` → a statement by itself
  - `IF (a>b) THEN add (a,b,sum)` → procedure call associated to another statement

Example of 3x3 signed vector multiplication

- Ex. 32 shows a 3x3 vector multiplication*

$$d = a \times b$$

$$d = [\, a0 \; a1 \; a2 \,] \times \begin{bmatrix} b0 \\ b1 \\ b2 \end{bmatrix}$$

$$d = a0\,b0 + a1\,b1 + a2\,b2$$

a is a row vector with 3 elements, and b a column vector with 3 elements ⇒ d is a row vector with 3 elements

Remember: $D = A\,B \Rightarrow$

$$d_{ij} = \sum_{h=1}^{n} a_{ih}\, b_{hj}$$

* adapted from: Nazeih Botros, *HDL with Digital Design – VHDL and Verilog*, Mercury Learning and Information, 2015

Example of 3x3 signed vector multiplication

- The example is composed by 3 procedures

```
procedure multiply_procedure (X, Y : in signed (3 downto 0);
    Z : out signed (7 downto 0));
```

  - find the partial products a0 b0, a1 b1, a2 b2 8 bits in size
  - based on the Booth algorithm ⟶ used to multiply two signed numbers in two's-complement format, according to VHDL standard
    - Andrew D. Booth. A signed binary multiplication technique. *The Quarterly Journal of Mechanics and Applied Mathematics*, Volume IV, Pt. 2, 1951
  - to prevent overflow `d` has 10 bits ≠ partial products size - 8 bits
    - different sizes ⇒ addition is not possible

Example of 3x3 signed vector multiplication

- possible solution: convert the partial to integers, perform the addition, revert the result to signed type

```
procedure signed_to_integer (binsg : in signed;
    M : in integer; int : out integer);
```

change the partial products to integers

```
procedure integer_to_signed (M, int : in integer;
    sbin : out signed);
```

change the resulting integer to binary

Example of 3x3 signed vector multiplication

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.my_procedures.all;

entity test_multiply_procedure is
    port (a0, a1, a2, b0, b1, b2 : in signed (3 downto 0);
          d : out signed (9 downto 0));
end test_multiply_procedure;
```

- user's `my_procedures` package is added to the design
- this package contains the 3 procedures

inputs A and B and output D

Example of 3x3 signed vector multiplication

```
architecture Behavioral of test_multiply_procedure is
begin
    process (a0, b0, a1, b1, a2, b2)
        variable tem0, tem1, tem2 : signed (7 downto 0);
        variable d_temp : signed (9 downto 0);
        variable temi0, temi1, temi2, temtotal : integer;

    begin
    -- Find the partial products a0b0, a1b1, a2b2
        multiply_procedure (a0, b0, tem0);
        multiply_procedure (a1, b1, tem1);
        multiply_procedure (a2, b2, tem2);
```

procedure is called 3 times to calculate each partial product

Example of 3x3 signed vector multiplication

```
    -- Change the partial products to integers
        signed_to_integer (tem0, 7, temi0);
        signed_to_integer (tem1, 7, temi1);
        signed_to_integer (tem2, 7, temi2);

    -- Find the total integer sum of partial products
        temtotal := temi0 + temi1 + temi2;

    -- Change the integer to binary
        integer_to_signed (9, temtotal, d_temp);

        d <= d_temp;
    end process;
end Behavioral;
```

procedure is called 3 times to convert each partial signed product to integer

performs the integer addition

procedure converts the result back to signed

FUNCTION vs PROCEDURE

FUNCTION	PROCEDURE
Parameters of mode `in` only; may not be changed inside the function → no parameters of mode `out`	All parameters may be modified inside the procedure
`wait` and `component` statements not allowed	`component` statements not allowed; `wait` statements allowed
only `CONSTANT` (by default) and `SIGNAL`; `VARIABLE` not allowed; any synthesizable data type, but with no range	`CONSTANT` (by default for `IN` parameters), `SIGNAL` or `VARIABLE` (by default for `OUT` and `INOUT` parameters); any synthesizable data type
Always return a value	May return none or one or more values depending on its `OUT` and `INOUT` parameters

Subprogram overloading

- Subprograms with the same name
- e.g.
  - `function add (a, b : std_logic_vector) return std_logic_vector;`
  - `function add (a, b : unsigned) return unsigned;`
- Both functions are overloaded ⟶ both have the same name
- When a call is made to overloaded functions or procedures ⟹ the called function or procedure is identified by
  - the number of parameters or
  - the type of parameters

Primitives and Macros

- Primitives ⟶ Xilinx components that are native to the targeted architecture
- Macros ⟶ primitives that are complex to instantiate by just using the primitives
  - synthesis tool automatically expand the unimacros to their underlying primitives
  - these elements are in the UniMacro library and the Xilinx Parameterized Macro library in Vivado
- Native to the targeted architecture ⟹ depend on the FPGA ⟶ varies with architecture

Primitives and Macros

- Primitives instantiated in a design → no optimization during synthesis → end up with exactly the same component in the back end, after the translation process
- Design Entry Methods
  - Instantiation
  - Inference
  - IP and IP Integrator Catalog
- Check FPGA's manufacturer documentation to know which primitives and macros are available in a specific FPGA

Primitives and Macros

- Information provided for each primitive and macro
  - name and description
  - schematic symbol (if any)
  - introduction
  - logic diagram (if any)
  - port descriptions
  - design entry method
  - available attributes (if any)
  - example instantiation code
  - links to additional information

Instantiation

- Direct instantiation of the component into the design
  - when full control over use, implementation, or placement of the individual blocks is required
- Requires the use of specific libraries
  - for primitives

```
Library UNISIM;
use UNISIM.vcomponents.all;
```

  - for macros

```
Library UNISIM;
use UNISIM.vcomponents.all;
library UNIMACRO;
use unimacro.Vcomponents.all;
```


Instantiation of Xilinx Parameterized Macros (XPM)

- For XPMs to be recognized by Vivado ⟶ they must be enabled using the XPM_LIBRARIES property on the project
  - a one-time setup on a project-by project basis
  - one or more of the available XPM types can be enabled
  - use `set_property` in the TCL Console command line
  - e. g. all types enabled

```
set_property XPM_LIBRARIES {XPM_CDC XPM_MEMORY XPM_FIFO}
[current_project]
```

  - e. g. only one type enabled

```
set_property XPM_LIBRARIES XPM_MEMORY [current_project]
```

Instantiation of Xilinx Parameterized Macros (XPM)

- After enabling it → XPM of the type enabled may be used in the design
- XPM use requires the use of the XPM library
  - `Library xpm;`
  - `use xpm.vcomponents.all;`
- Check documentation to see types and macros available
- VHDL and Verilog Instantiation Templates are provided whenever instantiation of a given XPM is available

Inference

- Provide a description of the function
- The synthesis tool interprets the HDL code to determine which hardware components to use to perform the function
- Vivado Design Suite provides Language Templates → pre-built code examples one can insert into his/her own design file

IP and IP Integrator Catalog

- Intellectual Property (IP) components → included in the design tool or acquired to other IP providers
- To instantiate IPs:
  - add a new source by selecting the IP Catalog
  - parameterize the module
- A 'Documentation' button located at the top left of the window enables access to the IP datasheet → contains a complete explanation of the IP operation and parameterization

IP and IP Integrator Catalog

- After parameterization, two files are generated:
  - a VHDL file that contains the "wrapper" code for simulation purposes
  - an HDL Instantiation Template file that contains the "instantiation" code to be used in the user's project

References

- Pong P. Chu, FPGA Prototyping by VHDL Examples: Xilinx Spartan-3 Version, 1st ed., John Wiley & Sons, Inc., 2008
- Fernando Pardo, José A. Boluda, *VHDL Lenguaje para síntesis y modelado de circuitos*, 3ª edición, RA-MA Editorial, 2011
- IEEE Std 1076™-2008 (Revision of IEEE Std 1076-2002), IEEE Standard VHDL Language Reference Manual, 2nd Printing, IEEE Computer Society, 2011
- J. Bhasker, *A VHDL Primer*, 3rd edition, Prentice-Hall, 1999
- John R. Fanchi, *Math Refresher for Scientists and Engineers*, 2nd edition, John Wiley & Sons, Inc., 2000
- B. Mealy, F. Tappero, *Free Range VHDL Book*, freerangefactory.org, 2016
- Nazeih Botros, *HDL with Digital Design – VHDL and Verilog*, Mercury Learning and Information, 2015
- Peter J. Ashenden, Jim Lewis, *VHDL-2008 - Just the New Stuff*, Morgan Kaufmann, 2008
- Sudhakar Yalamanchili, *VHDL – A Starter's Guide*, 2nd edition, Pearson, 2005
- Volney A. Pedroni, *Circuit Design with VHDL*, MIT Press, 2004
- Several User Guides and other documentation provided by Xilinx (http:\\www.xilinx.com)